

TradeMind Brain Build Book

Requirements, design and a module-by-module build plan for the TradeMind central intelligence brain. Each module can be built, switched on and checked on its own, and the brain keeps working with any module missing.

20 modules

8 brain steps

1 mandatory safety gate

6 build waves

Product	SwingTradeML · TradeMind intelligence platform
Source picture	“TradeMind · Complete Trading Intelligence Platform” v5, and the Living Architecture Review of 29 Sep 2026
Code checked	feature/plans-manager at 8c57127; production rollback point is tag v1 = 64fcb05
Settled	Two-set decision vocabulary with a market banner · up to 15 trading days, +8% / -4% · not-possible boxes listed as not planned · every module has a safe fallback, the risk gate is mandatory
Date	29 September 2026
Readers	The owner, who decides what to build next, and Claude, which builds it

How to use this book

This book turns the v5 platform picture into things we can build one at a time. It is written for two readers: the owner and Claude. Each section starts in plain words, and the technical detail follows.

If you are the owner

- Read Part A (four short sections) to see the whole idea.
- Use the module index to pick what to build next. The build plan in section 17 shows what must come first.
- Ask for one module at a time: *“Take M03 and finish it, then start the local server.”*
- Each module sheet ends with a green **“Done when you see”** box. That is your check.

If you are Claude, building a module

- Read section 10 (module contract) and section 12 (data contracts) first. They do not change per module.
- Build only what the module sheet lists. Never edit another module's internals; talk through contracts.
- Always write the fallback and the contract tests, even if the module is simple.
- Never touch the live trading path of version 1 unless the sheet says so (only M18 does).

How the book is organised

PART	WHAT IT ANSWERS	SECTIONS
A • Understand	What are we building, what already exists, and which rules shape every decision	1–4
B • Requirements	What the brain must do (functional) and how well (non-functional), and what is not planned	5–8
C • Solution design	How the brain works inside: the thinking pipeline, module contract, data shapes, storage, safety	9–16
D • Module catalogue	One sheet per module: purpose, inputs, outputs, processing, fallback, files, local check	M00–M19
E • Build plan	Order of work (serial and parallel), how to ask, how to run locally, testing, going live	17–21
F • Reference	Risks, a traceability table from every picture box to a module, and a glossary	22–24

Words used in this book

WORD	MEANING HERE
Brain	The new decision system. It reads data, thinks in 8 steps, and produces decisions with reasons.
Step	One of the 8 stages of thinking: perceive, state, recognise, remember, reason, risk, decide, learn.
Module	A piece of work you can ask for by its number (M00–M19). A module either fills a step or plugs into one.
Plug-in	A module that adds knowledge to a step without replacing it (for example, the sector brain adds sector strength to the state step).
Contract	The fixed shape of what goes into and comes out of a module. If the contract holds, modules can be swapped freely.
Fallback	The simple, safe answer a step gives when its module is off, broken or has stale data.
Shadow	A module runs and its output is recorded, but nothing acts on it. Used to prove a module before trusting it.
Version 1 (v1)	Today's running bot. It keeps trading as it does now until the owner chooses to switch (M18).

Contents

A Understand

- 1 What we are building
- 2 The picture, and our reading of it
- 3 Design principles
- 4 What exists today

B Requirements

- 5 Functional requirements
- 6 Non-functional requirements
- 7 Decision vocabulary and horizon
- 8 Not planned, and why

C Solution design

- 9 How the brain thinks
- 10 The module contract
- 11 Running with modules off
- 12 Data contracts
- 13 Storage
- 14 Orchestration and timing
- 15 Safety constitution
- 16 API and screens

D Module catalogue

- Module index
- M00 Brain core and module registry
- M01 Data gateway and quality score
- M02 Perception
- M03 State engine

- M04 Situation recognition
- M05 Memory and experience
- M06 Reasoning and models
- M07 Risk and safety gate
- M08 Decision engine
- M09 Learning loop
- M10 Market brain
- M11 Sector brain
- M12 Stock brain
- M13 News and events brain
- M14 Portfolio brain
- M15 Trade tracker and exit intelligence
- M16 Outputs, cards and alerts
- M17 Brain console and observability
- M18 Shadow run and go-live switch
- M19 User brain

E Build plan

- 17 Order of work: serial and parallel
- 18 How to ask for a module
- 19 Local server runbook
- 20 Testing
- 21 From shadow to live

F Reference

- 22 Risk register
- 23 Traceability: every box to a module
- 24 Glossary

1 • What we are building

A **brain** that looks at the market every day, understands the situation, remembers what happened in similar situations, and makes a clear, explained decision for each stock, while a safety layer keeps it inside hard limits.

Today's bot already does part of this. It reads prices, runs a machine-learning model, checks risk limits and places paper trades. But the thinking is spread across files and happens in one fixed chain: if one part is missing, the chain breaks. The brain puts the thinking into **eight named steps** with fixed contracts between them, so that each step can be improved, switched off or replaced on its own.

The three promises of this design

1 • Build one module at a time

Every module has its own sheet in Part D. You can ask for any module whose dependencies are done, and check it on your own computer when it is finished.

2 • Works with modules missing

Each step has a fallback. If the memory module is not built, the brain says “no past experience” and carries on. It is more careful, never broken.

3 • Safety is never optional

The risk and safety gate is the one mandatory module. If it is missing or fails, the brain says **NO NEW TRADES**. Existing exits keep running regardless.

What stays exactly as it is

- **Version 1 keeps trading.** The current strategies, exit loop (every 60 seconds), risk limits and Kite integration are not modified by any module except M18, which is the switch you control.
- **The locked trade rules.** Target +8%, stop −4%, up to 15 trading days, a 15% drawdown halt, a 25% sector cap and taking half off at the first target (+5%).
- **Paper first.** The brain runs in shadow until its record justifies trusting it, then goes through a manual-approval phase before any automatic live trading.

Settled decisions this book is built on

TOPIC	DECISION (29 SEP 2026)
Decision words	Two sets and a banner. New ideas: TRADE / WATCH / WAIT / AVOID . Holdings: HOLD / MONITOR / REDUCE / EXIT . Whole market: NORMAL / DEFENSIVE / NO NEW TRADES .
Holding period	Up to 15 trading days with the +8% / −4% rule. The brain's contracts carry a <code>horizon</code> field, so other horizons can be added later without redesign.
Not-possible boxes	Web scraper, alternative data and social sentiment, community and expert views, broker execution for other users, white-label and B2B: listed in section 8 as not planned, with no module sheets.
Flexibility	Every module has a contract and a safe fallback. The brain runs with any module off. The risk and safety gate cannot be skipped.

2 • The picture, and our reading of it

This is the owner's v5 platform picture. Everything in it is placed somewhere in this book: as a module, inside a module, as a design rule, or in the “not planned” list. Section 23 traces every box.

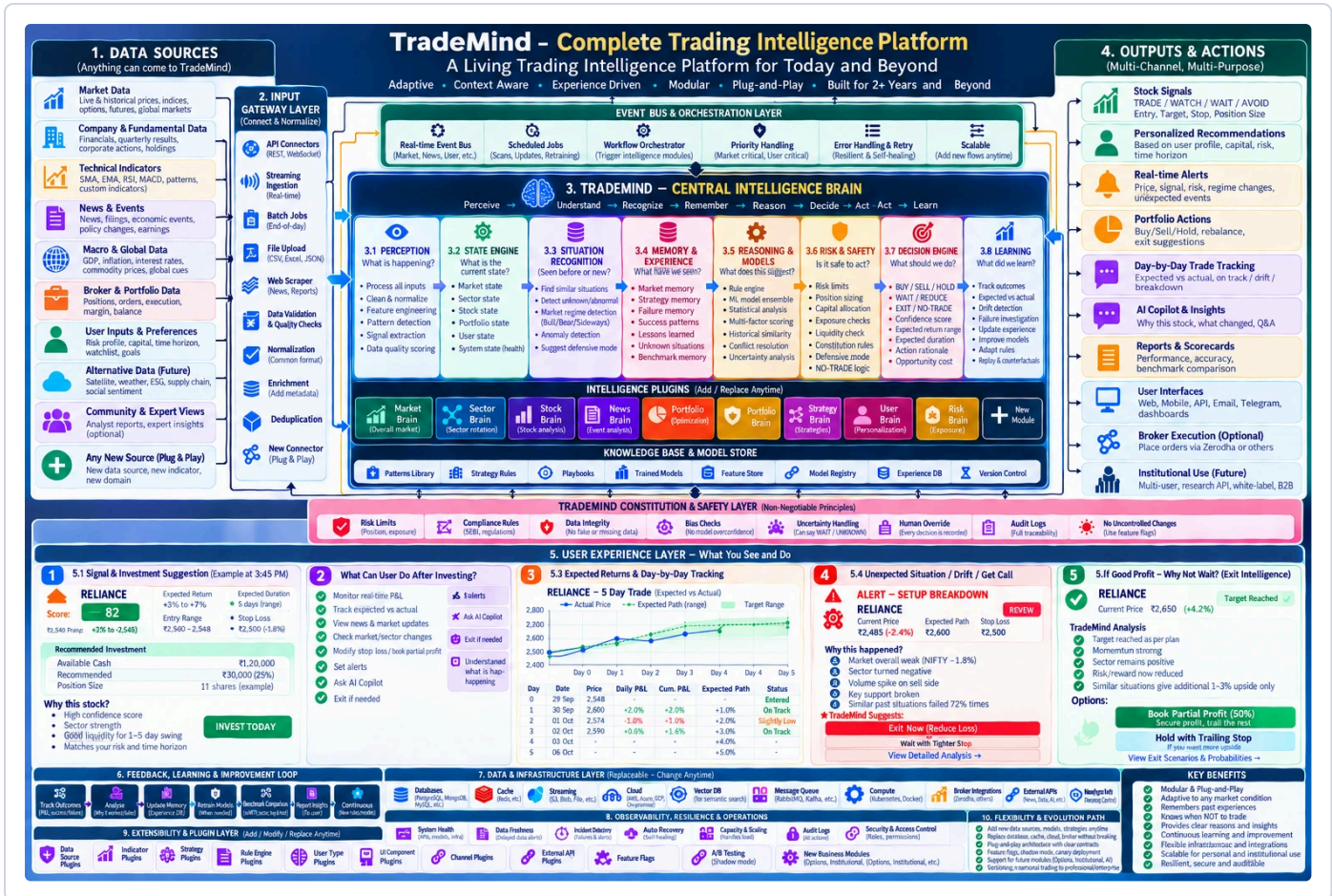


Figure 1. TradeMind · Complete Trading Intelligence Platform, v5 (owner's picture).

The same picture, reduced to what we build

The v5 picture has about 100 boxes. Many are the same idea seen from different sides. Reduced to its working parts, the platform is a straight line from data to decisions, with safety wrapped around the decision and learning feeding back.

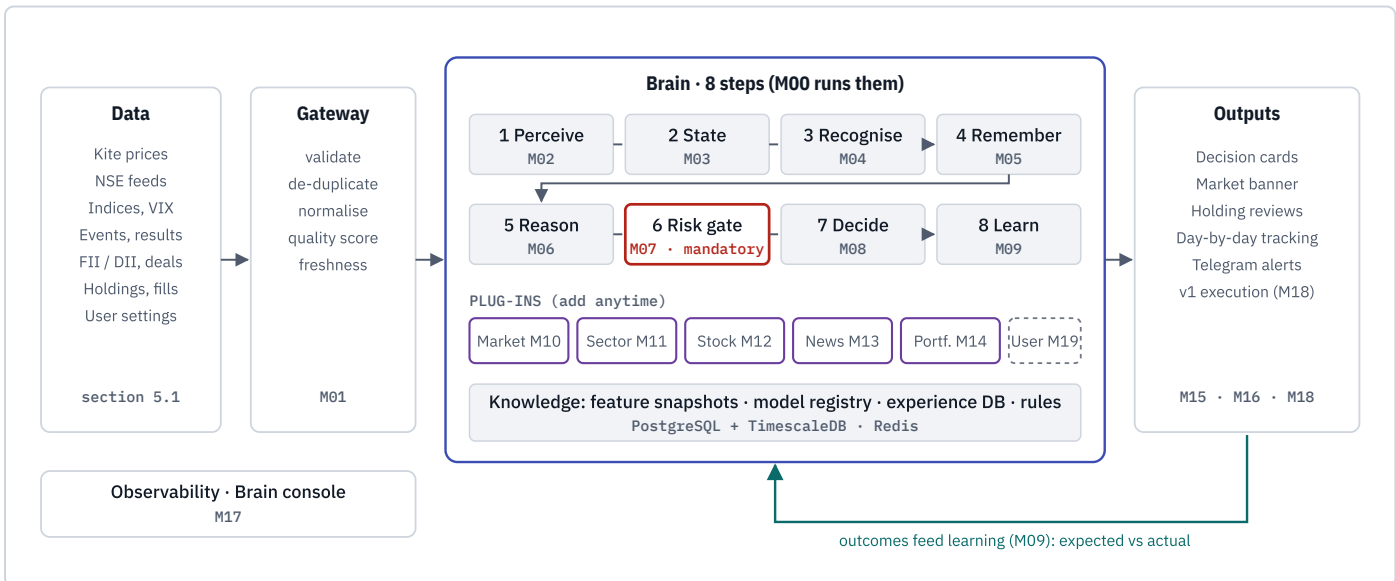


Figure 2. The v5 picture reduced to working parts. The red box is the only step that can never be switched off.

How the v5 layers map to this book

V5 LAYER	WHERE IT LIVES IN THIS BOOK

1 Data sources · 2 Input gateway	M01 data gateway and quality score. Existing ingestion is reused, and new sources arrive through a connector contract.
Event bus and orchestration	Section 14. The existing scheduler plus a brain run job first, and an event bus when there is a real need for one.
3 Central intelligence brain (3.1–3.8)	M02–M09, one module per step, run by M00.
Intelligence plug-ins	M10–M14 and M19. The two portfolio boxes are merged into M14; the strategy brain lives inside M06.
Knowledge base and model store	Section 13 (storage) and M05 (experience DB). The model registry already exists.
Constitution and safety	Section 15 and M07. Human override is the owner's halt and overrule; audit is a decision record.
4 Outputs · 5 User experience	M15 trade tracker, M16 outputs and cards.
6 Feedback and learning	M09. Learning proposes changes; the owner approves them.
7 Infrastructure · 8 Observability	Section 13 (keep it small) and M17 brain console.
9 Extensibility · 10 Flexibility path	Section 10 (module contract), section 11 (fallbacks) and M18 (shadow and gradual rollout).

PART A · UNDERSTAND

3 · Design principles

Twelve rules that every module follows. When a module sheet and a principle seem to disagree, the principle wins, and the module sheet gets corrected.

#	PRINCIPLE	WHAT IT MEANS IN PRACTICE
P1	Contracts first	Modules talk only through the data contracts in section 12. A module can be rewritten completely as long as its contract holds.
P2	Every step has a fallback	A step never raises an error to the brain. Off, failing, too slow or fed stale data means that step's fallback answer is used and the reason is recorded.
P3	Missing knowledge makes the brain more careful	A fallback can only make a decision more cautious (TRADE→WATCH→WAIT), never bolder. Missing modules can never create a trade.
P4	Safety is mandatory	M07 must run for any TRADE to leave the brain. Its absence means NO NEW TRADES. Exits never depend on the brain.
P5	Wrap, don't rewrite	Working code (features, models, risk checks, limits, exit policy) is called from the brain, not copied or rewritten.
P6	Version 1 is untouched	The brain runs beside v1. Only M18 connects the brain to execution, and only when the owner switches it.
P7	Shadow before trust	New modules start in shadow mode. They are switched on only after their recorded output has been checked.
P8	Only data known at the time	Every input carries an <code>as_of</code> time. Replays use only what was known then, which prevents look-ahead bias.
P9	One vocabulary	TRADE / WATCH / WAIT / AVOID for ideas, HOLD / MONITOR / REDUCE / EXIT for holdings, NORMAL / DEFENSIVE / NO NEW TRADES for the market. No other decision words anywhere.
P10	Every decision explains itself	A decision carries plain-English reasons and a trace of which modules ran, which fell back and why.
P11	Evidence, not promises	Expected returns are shown as a track record: “reached +8% in 41% of 120 similar signals”, never “you will make 8%”.
P12	Learning proposes, the owner approves	No module changes rules, thresholds or models by itself. It writes a proposal; the owner accepts or rejects it.

Why P3 matters most. The user wanted the brain to “work as intended” even if some modules are never started. P3 is what makes that safe: a brain with fewer modules is a more cautious brain, not a broken or reckless one.

PART A · UNDERSTAND

4 · What exists today

A large part of the brain already exists as working code. The brain modules wrap it. This table is the starting inventory; it was checked against the code on `8c57127`.

BRAIN STEP OR NEED	EXISTING CODE	WHAT IT ALREADY DOES	GAP THE BRAIN FILLS
Perceive	<code>ml/features.py</code> (464 lines), <code>ml/market_context.py</code>	53 indicators (RSI, MACD, ATR, ADX, MFI, OBV...), NIFTY, sector, VIX and breadth context	One dated snapshot per run; a data quality score
Data gateway	<code>services/ingestion.py</code> , <code>services/market_feeds.py</code>	Daily candles, bhavcopy with delivery, bulk and block deals, FII/DII, events, ASM/GSM lists; <code>feed_status()</code>	Quality score per input, and a freshness verdict the brain can read
State	<code>market_context.classify_regime</code> , <code>services/deployable.py</code>	Bullish/bearish trend, volatility level, deployable-capital %	Sector, stock, portfolio and system state in one object
Recognise	none	none	Situation labels, “unknown market” detection, defensive suggestion
Remember	signals, trades, <code>evaluate_signals</code>	Every signal and its outcome is stored	Episodes and “similar situations” lookup
Reason	<code>ml/predict.py</code> , <code>strategies/*</code> , <code>ml/calibration.py</code>	Model probabilities, ML swing, SMA crossover and long-term value strategies, calibration report	Opinions from many sources in one format; conflict handling
Risk	<code>services/risk.py</code> <code>check_entry()</code> , <code>services/limits.py</code> , <code>services/avoid.py</code> , <code>system_state.py</code>	Kill switch, cooldown, avoid lists, position and sector limits, sizing, drawdown halt, cash floor	Market-level NO NEW TRADES and DEFENSIVE modes; brain-wide gate
Decide	<code>services/engine.py</code> , <code>risk.rank_buy_candidates</code>	Runs strategies, ranks candidates, fills slots	The decision vocabulary, ranges, opportunity cost
Learn	<code>predict.evaluate_pending*</code> , <code>ml/train.py</code> , <code>ml/registry.py</code>	Outcome tracking, training, versioned models with rollback	Expected-vs-actual, drift, failure review, proposals
Exits	<code>services/exit_policy.py</code> , <code>execution.py</code> (1,571 lines), <code>job_check_exits</code>	Stops, targets, half-out, trailing stop, time stop, every 60 seconds	Day-by-day status (on track, drift, breakdown), reasons
Plug-in pattern	<code>strategies/base.py</code> <code>@register_strategy</code>	A working registry pattern with pure <code>evaluate()</code> functions	The same pattern for brain modules (M00)
Switches	<code>core/plans.py</code> , owner switch, <code>signals.is_shadow</code>	Plan gating, an owner switch, shadow signals	Per-module ON / SHADOW / OFF

Known live issues to keep visible. The brain must not bury these: the after-close order problem on the roadmap tracker, no barrier model on production yet, and v2 profile work waiting on the factor branch. Section 17 shows where they sit relative to the brain waves.

What the backend looks like today

FastAPI API with 18 routers under `/api/v1`, an APScheduler worker running about 20 jobs (quotes and exits every 60 seconds, ingest 15:40, predictions 15:42, scan 15:45 IST), PostgreSQL 17 with TimescaleDB, Redis, a React + Vite front end, all under Docker Compose. Production runs on one DigitalOcean droplet. The brain adds one package, `backend/src/swing_trade_ml/brain/`, and one front-end page. It adds no new infrastructure.

PART B · REQUIREMENTS

5 · Functional requirements

What the brain must do. Each requirement has an ID (BR = brain requirement), a priority, the v5 box it comes from, and the module that delivers it. Priority: **Must** = needed for the first working brain; **Should** = needed before going live; **Could** = valuable later.

5.1 Inputs and data

ID	REQUIREMENT	PRIORITY	V5 SOURCE	MODULE
BR-01	Read daily candles, indices, VIX and breadth for the watchlist and context symbols, stamped with the time they were known.	Must	1 Market data	M01, M02

BR-02	Read NSE feeds already stored: delivery %, bulk and block deals, FII/DII flows, corporate events, ASM/GSM restrictions.	Must	1 Company data, Macro	M01
BR-03	Score each input's quality (fresh, complete, plausible) from 0 to 1, per symbol and overall.	Must	2 Validation and quality	M01
BR-04	Refuse to decide TRADE on a stock whose data is stale or implausible; downgrade to WATCH with the reason.	Must	Constitution: data integrity	M01, M07
BR-05	Accept a new data source through a connector contract, without changing any brain step.	Should	1 Any new source · 2 New connector	M01
BR-06	Read the owner's holdings, open positions and cash (paper and real books) as portfolio input.	Must	1 Broker and portfolio	M03, M14
BR-07	Read user settings (risk level, capital, watchlist) as user input; personal advice is out of scope.	Could	1 User inputs	M19

5.2 Thinking: the eight steps

ID	REQUIREMENT	PRIORITY	V5 SOURCE	MODULE
BR-10	Run the eight steps in a fixed order for a given universe and time, producing one decision per stock, one review per holding and one market banner.	Must	3 Brain	M00
BR-11	Allow any module to be ON, SHADOW or OFF at run time, set by the owner without a deploy.	Must	9 Feature flags, 10 Flexibility	M00, M17
BR-12	Use the step's fallback when its module is off, errors, times out or reports stale data; record which fallback was used and why.	Must	Constitution · error handling	M00
BR-13	Build a feature snapshot per stock from existing indicators, dated and stored for replay.	Must	3.1 Perception	M02
BR-14	Describe the current state of market, sectors, each stock, the portfolio and system health.	Must	3.2 State engine	M03, M10, M11
BR-15	Label the situation (trend, pullback, breakout, correction, crash...), detect unknown or abnormal markets, and suggest defensive mode.	Should	3.3 Situation recognition	M04
BR-16	Find similar past situations and report what happened next (hit rate, typical path, sample size).	Should	3.4 Memory	M05
BR-17	Collect opinions from models, strategies and plug-ins in one format, with confidence and reasons, and resolve conflicts.	Must	3.5 Reasoning	M06
BR-18	Gate every new trade through the existing risk rules and the market-level mode; size it; enforce NO-TRADE.	Must	3.6 Risk and safety	M07
BR-19	Produce the decision word, entry zone, target, stop, size, horizon, return range with its record, and opportunity cost.	Must	3.7 Decision engine	M08
BR-20	Compare expected with actual, detect drift, investigate failures and write change proposals.	Should	3.8 Learning · 6 Feedback loop	M09

5.3 Plug-in intelligence

ID	REQUIREMENT	PRIORITY	V5 SOURCE	MODULE
BR-30	Market brain: regime, breadth, volatility, FII/DII flow, and a market mode (NORMAL / DEFENSIVE / NO NEW TRADES).	Must	Market brain	M10
BR-31	Sector brain: sector strength ranking and rotation, used as a state input and a tie-breaker.	Should	Sector brain	M11
BR-32	Stock brain: setup detection (pullback, breakout, base), delivery and deal signals, stock profile.	Should	Stock brain	M12
BR-33	News and events brain: results and ex-dates as risk windows; filings and big deals as events. No scraping.	Should	News brain	M13
BR-34	Portfolio brain: exposure, concentration, correlation and opportunity cost across held and candidate stocks.	Should	Portfolio (optimisation) + Portfolio brain	M14
BR-35	Strategy brain: know each strategy's record by regime, and weight its opinions accordingly.	Could	Strategy brain	M06
BR-36	User brain: personal views and sizing only; no personalised advice until legal review.	Could	User brain	M19

5.4 Outputs, tracking and control

ID	REQUIREMENT	PRIORITY	V5 SOURCE	MODULE
BR-40	Show a decision card per stock: word, one-line reason, levels, size, record, what could go wrong.	Must	4 Stock signals · 5.1	M16
BR-41	Show a market banner (NORMAL / DEFENSIVE / NO NEW TRADES) on every screen.	Must	Decision states	M16
BR-42	Track each open trade day by day against the typical path of similar trades: on track, drift or breakdown.	Should	5.3 · 4 Day-by-day tracking	M15
BR-43	Explain an unexpected move and suggest REDUCE or EXIT; report stops that were hit as “stop hit”, never as a choice.	Should	5.4	M15
BR-44	At a target, show evidence for booking half and trailing the rest.	Should	5.5 Exit intelligence	M15
BR-45	Send Telegram alerts for market mode changes, new TRADE ideas, drift and breakdowns.	Should	4 Real-time alerts	M16
BR-46	Give the owner a brain console: module switches, health, last run, and a full trace for any decision.	Must	8 Observability · Audit	M17
BR-47	Let the owner halt new trades or overrule any single decision; record who and why.	Must	Constitution: human override	M07, M17
BR-48	Run the brain in shadow beside v1, compare the two, and switch execution only with the owner's approval.	Must (before live)	9 A/B (shadow) · 10 Gradual rollout	M18

PART B · REQUIREMENTS

6 · Non-functional requirements

How well the brain must work. These are checked by tests (section 20) or by the console (M17).

ID	QUALITY	REQUIREMENT	HOW WE CHECK IT
NFR-01	Resilience	A failing module never stops a brain run. The run always completes with decisions or explicit fallbacks.	Fault-injection test: each module raises, times out, returns garbage
NFR-02	Safety	With M07 off or failing, zero TRADE decisions leave the brain.	Contract test on the runner; runs on every commit
NFR-03	Caution under missing modules	Turning any module off never raises the number of TRADE decisions on the same day's data.	Monotonicity test over a stored golden day (section 20)
NFR-04	Speed	A nightly run over 30 stocks finishes in under 60 seconds; a single-stock “why” run in under 3 seconds.	Timing stored per run; shown in the console
NFR-05	Per-module time limit	Each module has a time budget (default 10 s nightly, 1 s intraday). Over budget means fallback.	Runner enforces; test with a slow fake module
NFR-06	Reproducibility	Re-running the brain for a past date with the same module versions gives the same decisions.	Replay test on a stored date
NFR-07	No look-ahead	No module reads data stamped after the run's <code>as_of</code> .	Data access goes through one dated reader; test with future rows present
NFR-08	Traceability	Every decision stores inputs used, module versions, fallbacks, reasons and final word.	Console trace view; <code>brain_decisions</code> table
NFR-09	Isolation from v1	No brain module writes to orders, trades or positions, except M18 through the existing execution service.	Import-lint rule and a test that the brain package does not import the broker
NFR-10	Contract versioning	Contracts carry a version. A module declares which versions it reads and writes; mismatches are refused at start-up.	Registry check at start-up
NFR-11	Runs locally	Every module can be run and checked on the owner's Windows PC with Docker Postgres and Redis, with no paid feed needed.	The “Done when you see” box on every module sheet
NFR-12	Small infrastructure	No new services: PostgreSQL + TimescaleDB, Redis, Docker Compose. New infrastructure only on a measured need.	Design review per module
NFR-13	Security	Brain console and switches are owner-only. Broker tokens are never read by brain modules.	Route dependency check; test
NFR-14	Plain language	Every reason shown to the owner is a plain-English sentence without unexplained jargon.	Review of card text per module

Budgets at a glance

Nightly run

30 stocks · all modules · < 60 s total · 10 s per module budget · once after 15:45 IST.

Intraday check

Holdings only · steps 1, 2, 6, 7 · < 5 s · every 15 minutes in market hours (M15).

“Why” request

One stock · all modules · < 3 s · on demand from the console or a card.

PART B · REQUIREMENTS

7 · Decision vocabulary and horizon

One set of words, used by the brain, the database, the API, the screens and Telegram alike. The words are defined once in `brain/contracts.py` and nowhere else.

7.1 For a stock you don't own

WORD	MEANING	WHEN THE BRAIN USES IT	V1 SIGNAL IT MAPS TO
TRADE	A planned entry with levels and size	Reasoning is strong, risk gate allows, data is good, market mode is NORMAL (or DEFENSIVE with a smaller size)	BUY
WATCH	Interesting, but not yet	Good setup waiting for a trigger; or a TRADE downgraded because of data, confidence or a missing module	HOLD (no action)
WAIT	No edge right now	Weak or conflicting opinions; the default when nothing is known	HOLD
AVOID	Stay away for a stated reason	ASM/GSM list, results within the blackout window, breakdown, avoid list, poor liquidity	HOLD + reason

7.2 For a stock you hold

WORD	MEANING	WHEN THE BRAIN USES IT
HOLD	Keep, the plan is working	Price is inside the expected band of similar trades (on track)
MONITOR	Keep, but watch closely	Drift: below the band but above the stop; or context weakened
REDUCE	Take some off	First target reached (book half, trail the rest), or the market mode turned DEFENSIVE with a large position
EXIT	Leave the position	Breakdown, exit model says the edge is gone, or the time stop. If the stop was already hit, the card says “Stop hit: exited at ₹...”

We trade delivery (CNC). There is no short selling, so SELL only ever means leaving a holding; it is not a decision word.

7.3 For the whole market

MODE	EFFECT ON DECISIONS	TYPICAL TRIGGER (M10 DECIDES)
NORMAL	Full sizing within limits	Trend up or sideways; volatility normal or low; breadth healthy
DEFENSIVE	Fewer slots, smaller size, only the strongest TRADEs; REDUCE on large weak holdings	Trend down or volatility elevated; breadth weak; situation “unknown”
NO NEW TRADES	Every TRADE becomes WATCH; exits unaffected	Owner halt; drawdown halt (15%); M07 missing; crash situation; overall data quality below floor

7.4 Horizon and trade rules

FIELD	VALUE NOW	WHERE IT IS SET	CAN CHANGE LATER?
Horizon	Up to 15 trading days	<code>ML_PREDICTION_HORIZON_DAYS</code> , strategy params	Yes: every contract has a <code>horizon_days</code> field
Target / stop	+8% / -4%	Barrier label, strategy params	Only if replay proves setup-specific levels (owner decision)
First target action	At +5%: book half, trail the rest	<code>exit_policy.py</code>	Owner decision
Time stop	30 calendar days ($\approx$ 15 trading days + buffer)	Exit policy	Do not “fix” to 15; they already agree
Drawdown halt · sector cap	15% · 25%	<code>limits.py</code>	Owner decision

Showing expected return. The decision card never says “expected +3% to +7%” on its own. It says: *“Similar signals reached +8% before -4% in 41% of 120 cases within 15 trading days; the middle half ended between -2% and +6%.”* Where fewer than 30 similar cases exist, the card says so and shows the overall record instead.

PART B · REQUIREMENTS

8 · Not planned, and why

These v5 boxes are deliberately left out. They have no module sheet. If the situation changes (a licence, a legal opinion, a registration), they can be brought back as new plug-ins through the same contracts, because nothing in the design prevents it.

V5 BOX	WHY IT IS NOT PLANNED	WHAT WOULD HAVE TO CHANGE
Web scraper (news, reports)	Sites block it, it breaks often, and it goes against most sites' terms.	A licensed news feed or API with display rights
Alternative data (satellite, weather, ESG, supply chain)	High cost and licensing, little proven value for 15-day swings.	A specific, tested hypothesis worth the spend
Social sentiment	Noisy, easy to manipulate, costly to license.	A licensed feed and a replay test that shows value
Community and expert views, analyst reports	Hosting unregistered advice is a regulatory risk; analyst reports are licensed.	SEBI RA registration and content licences
Broker execution for other users	Placing orders for users brings algo-trading and broker-approval rules.	Legal review and broker approval
White-label, B2B, research API	Distributing our calls to other firms brings research-distribution rules.	Legal review; a business decision
MongoDB, MySQL, Kafka, Kubernetes, separate vector DB	Not needed at this size; each adds operations work.	A measured load problem (pgvector covers vectors)
“Self-evolving” learning that changes rules by itself	Unsafe with real money; breaks reproducibility.	Never: learning proposes, the owner approves (P12)

Also not in scope for this book: the Free / Plus / Pro plans, billing and sign-up screens. They are on the `feature/plans-manager` branch and have their own design. The brain's outputs are plan-agnostic; plan gating decides who sees which card.

9 • How the brain thinks

A brain run is like a careful trader's evening routine: look at the data, understand the market, recognise the situation, remember similar times, form opinions, check the rules, decide, and later learn from what happened.

Technically, a run passes one object, the **brain context**, through the eight steps. Each step reads what earlier steps wrote and adds its own part. Nothing is passed around informally: if it is not in the context, a later step cannot see it.

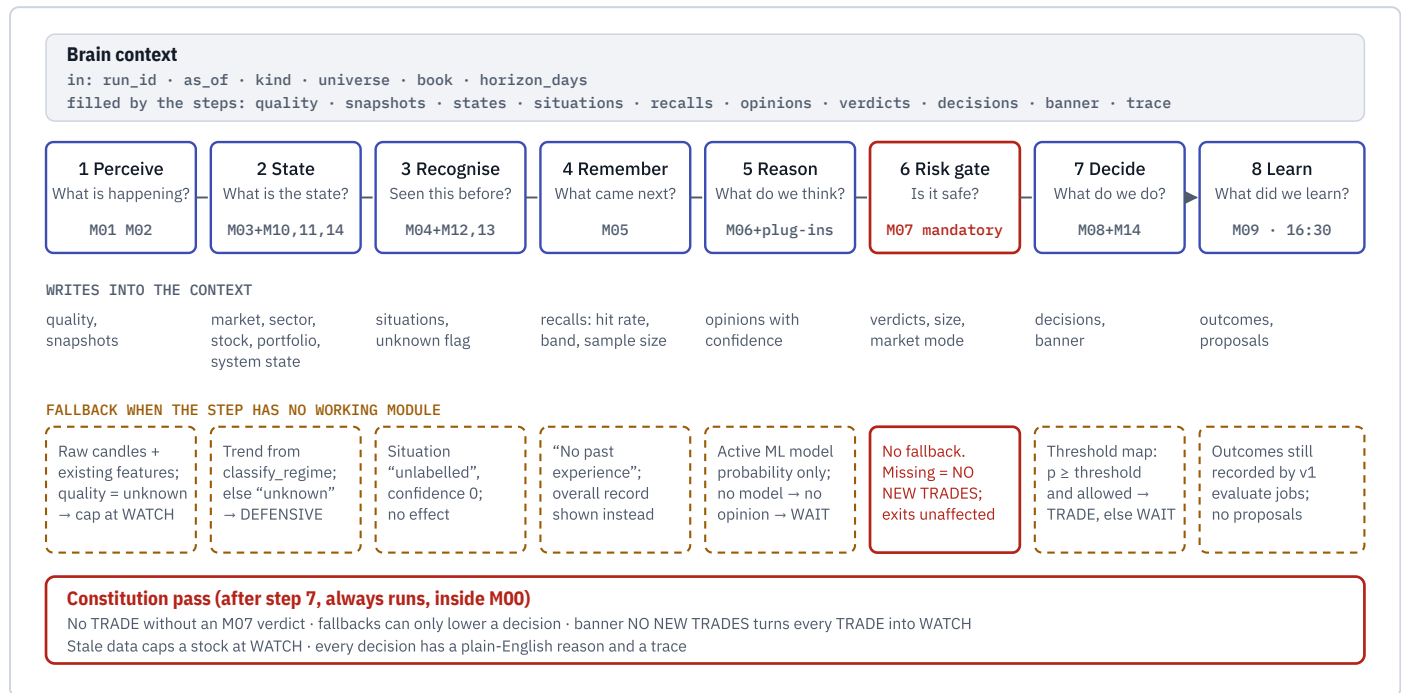


Figure 3. One brain run. Top row: the step and its modules (plug-ins marked +). Middle: what each step writes. Dashed amber: the fallback used when no module in that step works. Red: the safety gate and the constitution pass, which cannot be switched off.

9.1 The three kinds of run

RUN KIND	WHEN	UNIVERSE	STEPS	RESULT
nightly	15:50 IST on trading days	Watchlist (20–30) + holdings	1–7 (8 runs at 16:30 separately)	Tomorrow's decisions, banner, holding reviews
intraday	Every 15 min, 09:30–15:15	Holdings only	1, 2, 6, 7 (M15 adds tracking)	HOLD / MONITOR / REDUCE / EXIT updates, drift alerts
why	On demand	One stock	1–7	A full trace for the console or a card's "why?" link

9.2 Step modules and plug-ins

Each step has one **step module** that owns the step's result, and any number of **plug-ins** that add to it. For example, step 2 (state) is owned by M03, which builds the state object; the market brain (M10) and sector brain (M11) plug into it and fill in the market and sector parts. If M10 is off, M03 still produces a basic market state from the existing regime classifier. If M03 is off too, the step fallback gives "unknown" and the brain goes DEFENSIVE.

9.3 How the brain reaches v1 execution

The brain is registered as one more strategy in the existing registry: `@register_strategy class BrainStrategy` with `strategy_type = "brain"`. Its `evaluate()` reads the stored brain decision for that stock and returns a normal `SignalDecision` (TRADE → BUY with stop, target and horizon). Created as a **shadow** strategy, its signals are recorded and scored like any other but never

traded. M18 flips it from shadow to active when the owner approves. This reuses the whole execution, risk and exit machinery of v1 without changing it.

PART C · SOLUTION DESIGN

10 · The module contract

Every module, however simple, has the same outside shape. This is what makes “take this module and finish it” possible: the builder only needs this contract, the module sheet, and the data contracts.

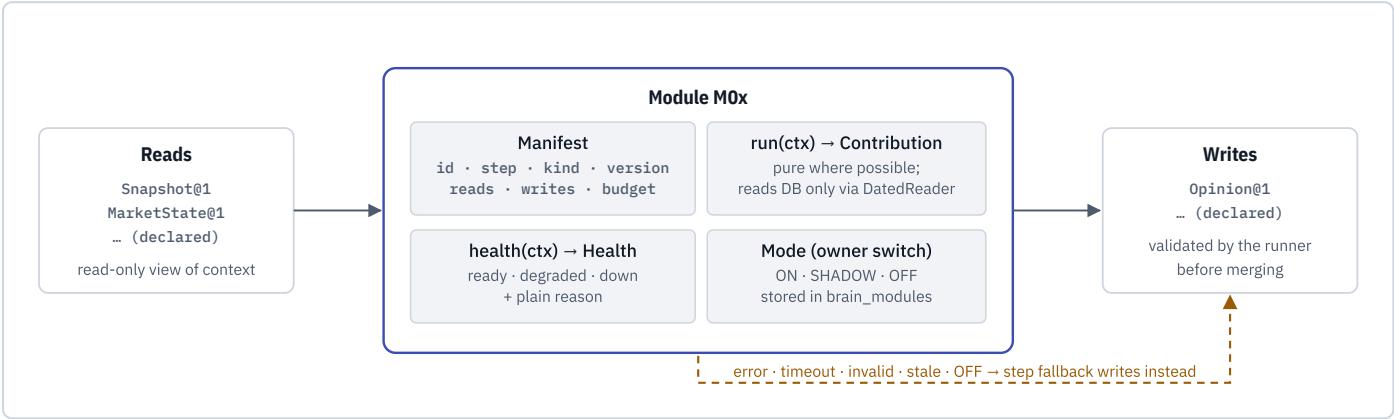


Figure 4. Anatomy of a module. The runner, not the module, decides when the fallback is used.

10.1 The interface

```
# backend/src/swing_trade_ml/brain/module.py
class Mode(StrEnum):
    ON = "on"          # output is used
    SHADOW = "shadow"  # runs, output recorded in the trace, not used
    OFF = "off"        # not run; step fallback applies

class Step(StrEnum):
    PERCEIVE = "perceive"; STATE = "state"; RECOGNISE = "recognise"; REMEMBER = "remember"
    REASON = "reason"; RISK = "risk"; DECIDE = "decide"; LEARN = "learn"

@dataclass(frozen=True, slots=True)
class Manifest:
    id: str          # "M03"
    name: str        # "State engine"
    step: Step
    kind: Literal["step", "plugin"]
    version: str      # module version, "1.0.0"
    reads: tuple[str, ...] # contract names with major version: ("Snapshot@1",)
    writes: tuple[str, ...] # ("MarketState@1", "StockState@1")
    budget_s: float = 10.0 # nightly time budget; intraday uses budget_s / 10
    mandatory: bool = False # only M07 sets this
    default_mode: Mode = Mode.SHADOW

class BrainModule(ABC):
    manifest: ClassVar[Manifest]

    @abstractmethod
    def run(self, ctx: ContextView) -> Contribution:
        """Read only what the manifest declares; return only what it declares."""

    def health(self, ctx: ContextView) -> Health:
        return Health.ready()

MODULE_REGISTRY: dict[str, type[BrainModule]] = {}

def register_module(cls: type[BrainModule]) -> type[BrainModule]:
    MODULE_REGISTRY[cls.manifest.id] = cls
    return cls
```

10.2 The runner

```
# brain/runner.py (simplified)
def run_brain(req: RunRequest, db: Session) -> BrainContext:
    ctx = BrainContext.start(req, reader=DatedReader(db, as_of=req.as_of))
    for step in STEPS_FOR[req.kind]:
        used = False
        for mod in registry.modules_for(step, db):          # step module first, then plug-ins
            if mod.mode is Mode.OFF:
                ctx.trace.skipped(mod, "switched off"); continue
            try:
                out = with_budget(mod.run, ctx.view_for(mod), mod.budget(req.kind))
                contracts.validate(out, mod.manifest.writes)
            except BudgetExceeded:
                ctx.trace.fallback(mod, "too slow"); continue
            except ContractError as e:
                ctx.trace.fallback(mod, f"bad output: {e}"); continue
            except Exception as e:
                ctx.trace.fallback(mod, f"error: {e!r}"); continue
            if mod.mode is Mode.SHADOW:
                ctx.trace.shadow(mod, out); continue
            ctx.merge(out); ctx.trace.used(mod); used = True
        if not used:
            ctx.merge(FALLBACKS[step](ctx)); ctx.trace.step_fallback(step)
    constitution.enforce(ctx)          # section 15; cannot be skipped
    store.save_run(db, ctx)
    return ctx
```

10.3 Rules for module builders

- **Read through the view.** `ctx.view_for(mod)` exposes only the contracts listed in `reads`. Database access goes through `DatedReader`, which never returns rows after `as_of`.
- **Write a Contribution.** A contribution is a small object holding lists of contract records. The runner merges it; modules never mutate the context directly.
- **Merge rules are per contract.** States: plug-ins fill their own sub-part (market, sector...). Opinions: appended. Verdicts: only M07. Decisions: only M08 (or the step fallback). Later-running plug-ins cannot overwrite an earlier module's fields.
- **No side effects in run().** No orders, no Telegram, no writes to v1 tables. Storing the run is the runner's job. Alerts are M16's job, after the run.
- **Contract versions.** A breaking change to a contract bumps its major version (`Opinion@2`). The registry refuses to start a module whose `reads` are not produced by any module or fallback.
- **Folder per module.** `brain/modules/m03_state/` with `module.py`, `logic.py` (pure functions), and `tests/`.

PART C · SOLUTION DESIGN

11 · Running with modules off

This section shows, concretely, what the brain does when modules are missing. It is the answer to “even if I don't start a few modules, the brain should work as intended”.

11.1 The minimum brain

The brain can produce real decisions with only **M00 (core)**, **M07 (risk gate)** and the step fallbacks. In that state it behaves roughly like v1 today: the active ML model's probability, the existing risk checks, and a threshold. Every module added after that makes it smarter. None is required for it to run.

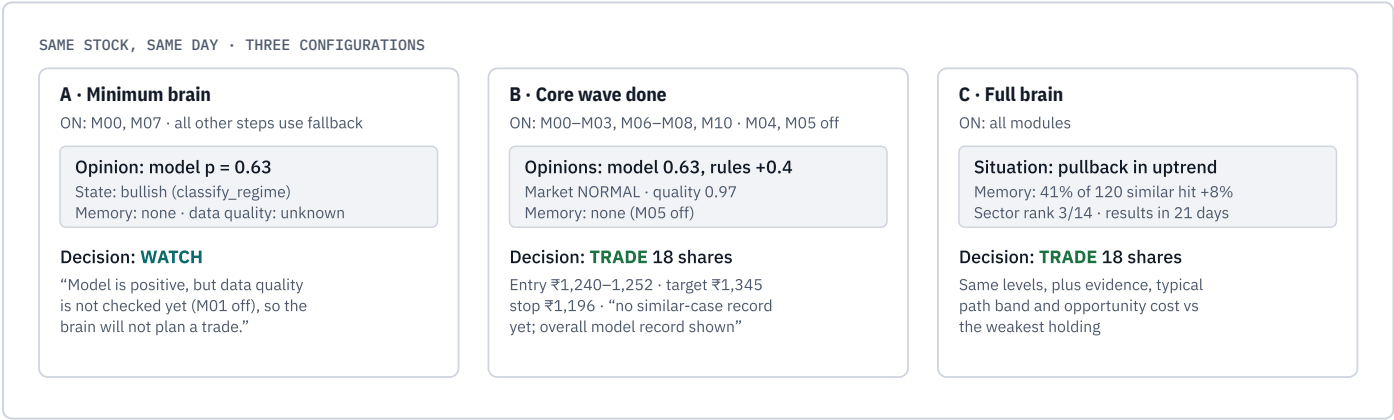


Figure 5. Illustrative example (made-up stock and numbers). Missing modules make the brain more careful and less informative, never reckless.

11.2 Fallback matrix

IF THIS IS OFF OR FAILING...	...THE BRAIN DOES THIS	EFFECT ON DECISIONS
M01 Data gateway	Quality = “unknown” for all symbols	All TRADE capped at WATCH (P3), banner reason “data not checked”
M02 Perception	Fallback computes the snapshot with <code>latest_feature_row()</code> directly	None, only no stored snapshot for replay
M03 State engine	Market trend from <code>classify_regime</code> ; other states empty	If the trend is unknown → DEFENSIVE
M04 Situations	Situation “unlabelled”, confidence 0	No boost, no defensive suggestion from situations
M05 Memory	“No past experience”	Card shows the model's overall record instead of similar cases

M06 Reasoning	Active ML model probability only; if no active model, no opinion	No opinion → WAIT
M07 Risk gate	No fallback	Banner NO NEW TRADES; every TRADE → WATCH; holdings still reviewed; v1 exits unaffected
M08 Decision engine	Threshold map: probability ≥ strategy threshold and allowed → TRADE; else WAIT. Holdings → HOLD	No ranges, no opportunity cost
M09 Learning	Nothing proposed; v1 evaluation jobs still record outcomes	None on today's decisions
M10 Market brain	M03 builds a basic market state; mode from trend and drawdown only	DEFENSIVE less precise
M11 Sector brain	No sector ranking	No sector tie-breaker; the 25% sector cap still applies in M07
M12 Stock brain	No setup labels	Card reason comes from the model only
M13 News brain	No event windows beyond the existing avoid list	Results blackout only as far as v1's avoid list covers it
M14 Portfolio brain	No correlation or opportunity-cost ranking	Ranking falls back to <code>rank_buy_candidates</code>
M15 Trade tracker	No day-by-day status	Holdings shown HOLD unless M07/v1 exits act
M16 Outputs	Decisions visible only in the console	No cards or alerts
M17 Console	Modes set from <code>brain.yaml</code> defaults	No owner switching without a restart

11.3 The monotonic rule

The constitution pass (section 15) enforces one simple ordering so that no combination of modules can produce a bolder decision than the full brain would allow:

AVOID < WAIT < WATCH < TRADE · EXIT < REDUCE < MONITOR < HOLD

A fallback or a failing check may move a decision to the left, never to the right. A test (NFR-03) proves this on a stored day by switching each module off in turn and comparing.

PART C · SOLUTION DESIGN

12 · Data contracts

The shapes that move between modules. They live in `brain/contracts.py` as frozen dataclasses (or Pydantic models for the API), each with a name and major version. Money is in rupees as `float`; dates are IST; every record carries `as_of`.

CONTRACT	WRITTEN BY	KEY FIELDS
Run		
RunRequest@1	scheduler, API	<code>run_id, kind (nightly intraday why), as_of, universe[symbol], book (paper real), horizon_days=15</code>
TraceEvent@1	runner	<code>module_id, step, status (used shadow fallback skipped), reason, ms, version</code>
Step 1 · Perceive		
DataQuality@1	M01	<code>symbol "*", score 0-1, last_bar_date, fresh, issues[{code, text}]</code>
Snapshot@1	M02 / fallback	<code>symbol, as_of, close, atr_14, adv_inr_20, features{name: value}, feature_set_version</code>
Step 2 · State		
MarketState@1	M03, M10	<code>trend (up down sideways unknown), volatility (low normal elevated unknown), breadth_pct, fii_net_5d_cr, vix, mode (NORMAL DEFENSIVE NO_NEW_TRADES), reasons[]</code>
SectorState@1	M11	<code>sector, rank, of_total, strength_20d, rotation (leading improving weakening lagging)</code>
StockState@1	M03, M12	<code>symbol, trend, rel_strength_vs_nifty, dist_from_52w_high_pct, delivery_signal, deals[], restrictions[]</code>
PortfolioState@1	M03, M14	<code>book, value, cash, positions[{symbol, qty, avg, pnl_pct, days_held}], sector_exposure{ }, drawdown_pct, free_slots</code>
SystemState@1	M03	<code>entries_halted, halt_reason, exits_disabled, broker_ok, quality_overall</code>
Steps 3–5 · Recognise, Remember, Reason		

Situation@1	M04, M12, M13	scope (market sector stock), subject, label, confidence, is_unknown, suggest_defensive, evidence[]
Recall@1	M05	symbol, query (situation + state key), n_similar, hit_rate, median_return, p25, p75, median_days, typical_path[{day, p25, p50, p75}], examples[ids]
Opinion@1	M06, plug-ins	source, symbol, stance (-1..+1), probability?, confidence 0-1, horizon_days, reasons[]
Steps 6–8 • Risk, Decide, Learn		
RiskVerdict@1	M07 only	symbol, allowed, max_qty, rule, reason, amount_inr, mode_applied
Decision@1	M08 / fallback	symbol, kind (idea holding), word, entry_low, entry_high, target, stop, qty, horizon_days, evidence_text, reasons[], downgraded_from?, downgrade_reason?, confidence
Banner@1	M08 / constitution	mode, headline, reasons[]
TrackPoint@1	M15	symbol, day_n, price, ret_pct, band_p25, band_p75, status (on_track drift breakdown), word
Outcome@1	M09	decision_id, hit (target stop time open), ret_pct, days, max_up, max_down
Proposal@1	M09	id, kind (threshold model rule module_mode), change, evidence, status (open accepted rejected)

12.1 Example: one decision record

```

{
  "contract": "Decision@1",
  "symbol": "EXAMPLE", "kind": "idea", "word": "TRADE",
  "entry_low": 1240.0, "entry_high": 1252.0, "target": 1345.0, "stop": 1196.0,
  "qty": 18, "horizon_days": 15, "confidence": 0.63,
  "evidence_text": "Similar signals reached +8% before -4% in 41% of 120 cases within 15 trading days.",
  "reasons": ["Pullback to the 20-day average inside an uptrend",
    "Sector ranked 3 of 14 and improving",
    "Delivery above its 20-day average for 3 days"],
  "downgraded_from": null,
  "trace_ref": "run 2026-10-06T15:50 / 14 events"
}

```

Illustrative values. Real records are written by M08 and stored in `brain_decisions`.

PART C • SOLUTION DESIGN

13 • Storage

Seven new tables, all in the existing PostgreSQL database, created through Alembic migrations owned by the module that needs them. No existing table changes shape. Snapshot and track tables are TimescaleDB hypertables because they grow daily.

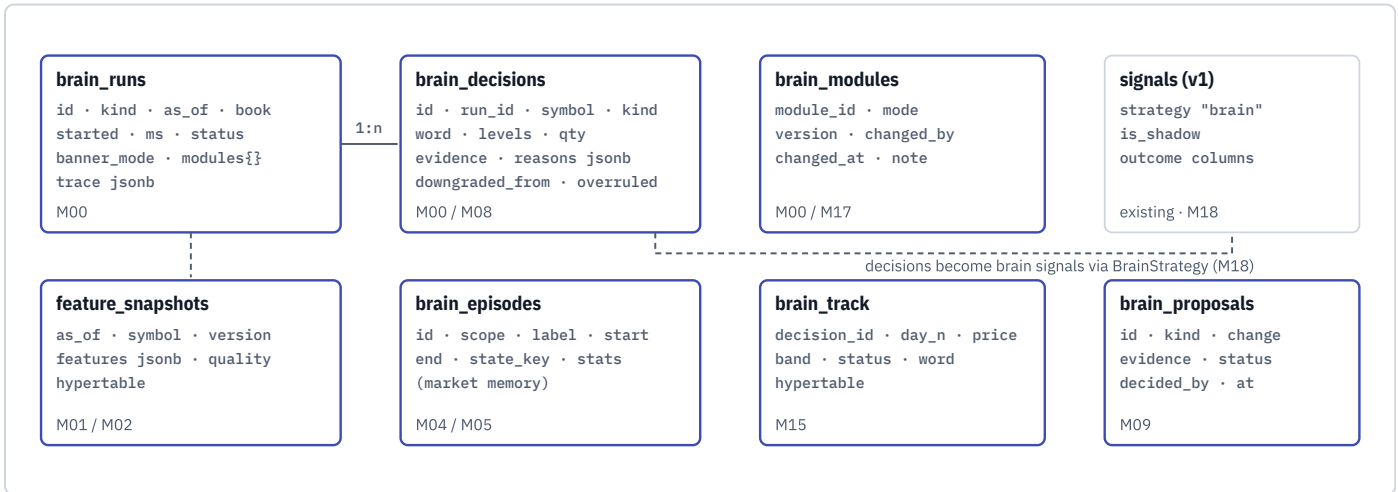


Figure 6. New tables (blue outline) and the one link to an existing table. Dashed lines are references, not foreign keys, where the rows are written at different times.

13.1 Retention and size

TABLE	ROWS PER DAY (30 STOCKS)	KEEP	NOTE
brain_runs	~27 (1 nightly + 26 intraday)	Nightly forever; intraday 90 days	Trace JSON ≈ 10–30 KB each
brain_decisions	~35 nightly + holdings intraday	Forever	This is the audit trail (BR-46, NFR-08)
feature_snapshots	~35	Forever	Enables exact replays and prevents look-ahead
brain_track	open positions × intraday runs	Forever (daily close rows); intraday rows 90 days	Compressed chunks after 30 days
brain_episodes, brain_proposals, brain_modules	few	Forever	Small

13.2 Configuration

Defaults live in `backend/src/swing_trade_ml/brain/brain.yaml` (module default modes, budgets, thresholds for the fallback decision map, DEFENSIVE sizing factor). The owner's switches in `brain_modules` override the file. An environment flag `BRAIN_ENABLED` (default `false` on production, `true` locally) turns the brain's scheduled jobs on or off as a whole.

PART C · SOLUTION DESIGN

14 · Orchestration and timing

The brain uses the existing APScheduler worker. It does not need a separate event bus to begin with: its jobs run after v1's jobs have fetched the data. An event bus is added only when a real-time need appears (for example Kite WebSocket ticks), and then behind the same runner.

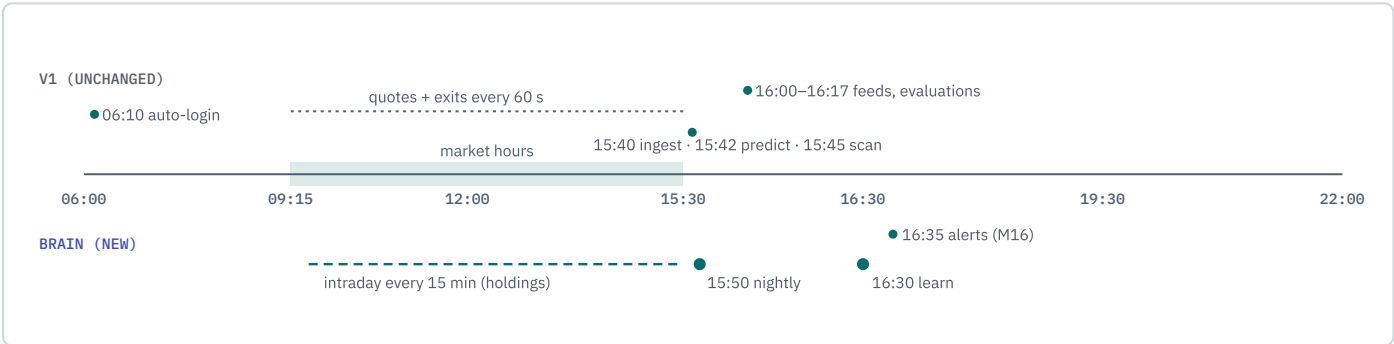


Figure 7. A trading day (IST). Brain jobs run after v1 has the data and never compete with the 60-second exit loop.

JOB ID	TRIGGER (IST, WEEKDAYS)	DOES	MODULE
brain_nightly	15:50	<code>run_brain(kind="nightly")</code> for watchlist + holdings, paper book (and real book if enabled)	M00
brain_intraday	09:30–15:15 every 15 min	Holdings-only run, steps 1, 2, 6, 7 + tracking	M00, M15
brain_learn	16:30	Outcomes, drift, proposals	M09
brain_alerts	16:35 and after each intraday run	Send what changed: banner, new TRADE, drift, breakdown	M16
brain_housekeeping	Sun 08:30	Drop old intraday runs, compress hypertables	M17

14.1 Priorities

- 1. **Exits first.** v1's `check_exits` keeps its own job and never waits for the brain.
- 2. **Owner actions next.** Halts and overrules are applied immediately and also stored in the next run.

3. **Runs never overlap.** A run lock (Redis key `brain:lock:{kind}`) prevents two runs of the same kind; a late run is skipped and logged, not queued.
4. **Retries.** A failed nightly run retries once after 5 minutes, then alerts the owner. Module errors are not retried within a run; the fallback is used.

PART C · SOLUTION DESIGN

15 · Safety constitution

The non-negotiable rules. They are enforced in code in two places: inside M07 (per trade) and in the constitution pass inside M00 (over the whole run). Neither can be switched off from the console.

#	RULE	ENFORCED BY	TEST
C1	No TRADE leaves the brain without an allowed M07 verdict for that stock.	Constitution pass	Run with M07 off → 0 TRADE
C2	A missing or failing M07 sets the banner to NO NEW TRADES.	Constitution pass	Fault injection
C3	Existing v1 limits apply unchanged: kill switch, cooldown, avoid list, position count, sector cap 25%, small-company budget, liquidity, cash floor, drawdown halt 15%.	M07 calls <code>risk.check_entry()</code>	Existing risk tests + adapter test
C4	Decisions only move toward caution when data, modules or checks are missing (section 11.3).	Constitution pass	Monotonicity test (NFR-03)
C5	Stale or implausible data caps a stock at WATCH.	M01 → constitution	Stale-data fixture
C6	A stop is never loosened. The app's stop only moves up (trailing).	M08, M15	Property test
C7	A stop that has been hit is reported as hit, never offered as a choice.	M15	Example 5.4 fixture
C8	The owner can halt new trades, disable a module or overrule a decision at any time; each action is recorded with who and why.	M17, <code>system_state</code>	API tests
C9	No module changes thresholds, rules or models by itself. Changes go through proposals and owner approval.	Code review; M09 writes only proposals	Import-lint: M09 cannot import registry writers
C10	No brain module places, modifies or cancels orders. Only v1 execution does, and only for the brain strategy after M18 is switched.	Package boundary	Import-lint: <code>brain</code> must not import <code>brokers</code> or <code>services.execution</code>
C11	Every decision has at least one plain-English reason and a trace reference.	Contract validation	Contract test
C12	Shown returns are records with sample size, never promises.	M08 evidence text, M16 card	Text lint for “will”, “guaranteed”

15.1 Human override

Halt

Existing `system_state.halt()`. The brain reads it in step 2 and sets NO NEW TRADES. Exits continue.

Switch a module

Console sets ON / SHADOW / OFF in `brain_modules`. M07 has no OFF button; the runner treats any attempt as refused.

Overrule a decision

`POST /brain/decisions/{id}/overrule` with a new word (more cautious only) and a reason. Stored on the decision and shown on the card.

PART C · SOLUTION DESIGN

16 · API and screens

All brain endpoints sit under `/api/v1/brain` in a new router `api/v1/endpoints/brain.py`, protected like the other routers. Owner-only routes use the existing owner dependency.

METHOD AND PATH	WHO	RETURNS / DOES	MODULE
GET <code>/brain/modules</code>	owner	Every module: manifest, mode, health, last status	M00
PUT <code>/brain/modules/{id}</code>	owner	Set mode (ON / SHADOW / OFF) with a note; M07 cannot be set OFF	M00
POST <code>/brain/runs</code>	owner	Start a run: <code>{kind, symbols?, as_of?}</code> ; returns run id and summary	M00
GET <code>/brain/runs/latest?kind=</code>	signed in	Latest run summary: banner, counts per word, timing	M00
GET <code>/brain/runs/{id}/trace</code>	owner	Every trace event of the run	M17
GET <code>/brain/decisions?run_id=&word=</code>	signed in	Decision cards	M16
GET <code>/brain/decisions/{id}</code>	signed in	One decision with evidence and a trace summary	M16
POST <code>/brain/decisions/{id}/overrule</code>	owner	Overrule with a more cautious word and reason	M17
GET <code>/brain/banner</code>	signed in	Current market mode and reasons	M16
GET <code>/brain/track/{symbol}</code>	signed in	Day-by-day track points and band	M15
GET <code>/brain/why/{symbol}</code>	signed in	Runs a <code>why</code> run; returns decision and trace	M00
GET/POST <code>/brain/proposals</code>	owner	List; accept or reject a proposal	M09
GET <code>/brain/health</code>	owner	Data freshness, module health, last run status	M17

16.1 Screens

Brain console (owner, M17)

Route `/brain`. Module list with switches and health dots, last runs, a “Run now” button, and a trace viewer that shows, for any decision, which modules ran, which fell back and why.

Market banner (M16)

A thin strip under the top bar on every page: NORMAL (quiet), DEFENSIVE (amber), NO NEW TRADES (red), with one sentence of reason.

Decision card (M16)

Word, one-line reason, entry zone, target, stop, size, evidence sentence, “what could go wrong”, and “why?” link to the trace. Buttons: **Plan this trade**, **Add to watchlist**. No “Invest today”.

Holding tracker (M15)

Per holding: small chart of price vs the band of similar trades, day counter, status word (on track, drift, breakdown) and the holding decision.

All screen text follows the plain-English rule (NFR-14): terms like ATR or percentile appear only behind a “details” tap.

Module index

Twenty modules. The **wave** is the earliest point a module can be built (section 17). Effort: **S** ≈ one working session, **M** ≈ two to three, **L** ≈ four or more. Every sheet follows the same layout: plain words, requirements covered, inputs and outputs, processing, fallback, reused code, new code, data, API and screens, dependencies, tests, and a green “Done when you see” check.

ID	MODULE	KIND	STEP	WAVE	EFFORT	DEPENDS ON
M00	Brain core and module registry	Core	All steps (runner)	0	L	Nothing. Must be first.
M01	Data gateway and quality score	Step module	1 Perceive	1	M	M00
M02	Perception	Step module	1 Perceive	1	S	M00
M03	State engine	Step module	2 State	2	M	M00 (M02 recommended)
M04	Situation recognition	Step module	3 Recognise	3	M	M03
M05	Memory and experience	Step module	4 Remember	4	L	M04 (for labels; can start with trend buckets only)
M06	Reasoning and models	Step module	5 Reason	1	M	M00
M07	Risk and safety gate	Mandatory	6 Risk	1	M	M00
M08	Decision engine	Step module	7 Decide	2	M	M00, M07 (M06 recommended)
M09	Learning loop	Step module	8 Learn	5	L	M08 and about 4 weeks of stored decisions
M10	Market brain	Plug-in	2 State (+ opinions)	2	M	M03
M11	Sector brain	Plug-in	2 State (+ opinions)	3	S	M03
M12	Stock brain	Plug-in	3 Recognise (+ opinions)	3	M	M02, M03
M13	News and events brain	Plug-in	3 Recognise (+ risk windows)	3	M	M01 (feed freshness), M03
M14	Portfolio brain	Plug-in	2 State + 7 Decide	4	M	M03, M08
M15	Trade tracker and exit intelligence	Step module	Intraday · 7 Decide (holdings)	4	M	M08 (M05 for real bands)
M16	Outputs, cards and alerts	Step module	After the run	2	M	M08
M17	Brain console and observability	Operations	Operations	1	M	M00
M18	Shadow run and go-live switch	Core	Connection to v1 execution	5	M	M08, M16, M17 and at least 30 resolved shadow decisions for stage 2
M19	User brain	Later	Inputs + 7 Decide	Later	M	M14, the plans branch, legal review

Minimum brain = M00 + M07. **First useful brain** (real TRADE ideas in shadow) = M00, M01, M02, M06, M07, M08, M03, M10, M16, M17. Everything else adds understanding, evidence and tracking.

M00 Brain core and module registry

Core

Wave 0

Effort L

The skeleton of the brain. It defines the eight steps, the data contracts, the module registry and the runner that calls modules, applies fallbacks and enforces the constitution. After M00, the brain already runs end to end using only fallbacks and a stub risk gate that always says NO NEW TRADES.

Brain step	All steps (runner)
Covers	BR-10, BR-11, BR-12 · NFR-01, 02, 05, 06, 08, 09, 10 · v5: 3 Brain, Event bus (orchestrator), 9 Feature flags, 10 Contracts
Reads (input)	RunRequest@1
Writes (output)	TraceEvent@1, the whole BrainContext, brain_runs and brain_decisions rows
Processing	- Package brain/: contracts.py, module.py (Manifest, Mode, Step, BrainModule, registry), context.py (BrainContext, ContextView, Contribution, merge rules), reader.py (DatedReader), runner.py, fallbacks.py (one function per step), constitution.py, store.py. - Runner exactly as in section 10.2, with per-module time budgets (thread with timeout) and contract validation. - Constitution pass C1, C2, C4, C5, C11 (section 15). - Step fallbacks for all 8 steps, as in the fallback matrix (section 11.2). The risk step's “fallback” is a hard NO NEW TRADES. - CLI: swingtrade brain run modules set why replay. - Scheduler jobs brain_nightly and brain_intraday, registered only when BRAIN_ENABLED=true. - Minimal API: GET /brain/modules, PUT /brain/modules/{id}, POST /brain/runs, GET /brain/runs/latest, GET /brain/why/{symbol}.
If switched off	Not applicable: M00 is the brain. If M00 itself fails, the job logs the error and alerts the owner; v1 is unaffected because nothing in v1 depends on the brain.
Reuses	strategies/base.py registry pattern, workers/scheduler.py, db/session.py, api/deps.py owner dependency, cli.py subcommand style.
New code	brain/ package (≈ 900 lines), api/v1/endpoints/brain.py, CLI subcommands, brain/brain.yaml.
Data	Migration: brain_runs, brain_decisions, brain_modules.
API and screens	Endpoints above. No screen yet (M17 adds the console).
Depends on	Nothing. Must be first.
Can be built with	Nothing; every other module needs it.
Tests	- Runner with fake modules: used, shadow, off, error, timeout, bad output → correct trace and fallback. - C1/C2: with no M07 registered, a fake 0.99 opinion still yields 0 TRADE and banner NO NEW TRADES. - DatedReader never returns rows after as_of (fixture with future rows). - Replay determinism: same inputs → identical decisions JSON.

Done when you see

1. Start the local stack (section 19).
2. Run swingtrade brain modules: lists M00 and 8 steps, all using fallback.
3. Run swingtrade brain run --kind nightly.
4. Open http://localhost:8000/api/v1/brain/runs/latest.

Expected: A finished run with a decision for every watchlist stock, all **WATCH** or **WAIT**, banner **NO NEW TRADES** with the reason “risk gate not installed”, and a trace showing 8 step fallbacks. That is the correct, safe behaviour of an empty brain.

M01 Data gateway and quality score

Step module

Wave 1

Effort M

Checks that the data the brain is about to use is fresh, complete and believable, and gives each stock a quality score. A stock with bad data can never become a TRADE. It also defines how a new data source is plugged in later.

Brain step	1 Perceive
Covers	BR-01, BR-02, BR-03, BR-04, BR-05 · v5: 2 Input gateway (validation, normalisation, dedupe, new connector), 8 Data freshness, Constitution: data integrity
Reads (input)	Candles, index, VIX, feeds via <code>DatedReader</code>
Writes (output)	<code>DataQuality@1</code> per symbol and overall (<code>symbol="★"</code>)
Processing	1. Freshness: last bar date equals the latest expected trading day (uses <code>core/holidays.py</code> and <code>market_feeds.latest_expected_delivery_day</code>). 2. Completeness: at least the strategy's <code>min_bars_required</code>; no gap longer than 3 trading days in the last 60. 3. Plausibility: one-day move beyond 20% without a corporate action → flag; zero volume → flag; high < low → flag. 4. Feed status: reads <code>market_feeds.feed_status()</code> for delivery, deals, FII/DII and events freshness. 5. Score = 1.0 minus weighted penalties; below 0.6 marks the symbol not fresh (cap at WATCH). 6. Connector contract <code>brain/connectors/base.py</code>: <code>fetch(as_of) → rows</code>, <code>validate(rows) → issues</code>, <code>store(rows)</code>. Existing ingestion is wrapped as the first connector; no new feed is added in this module.
If switched off	Quality “unknown” for every symbol. The constitution caps all TRADE at WATCH with the reason “data not checked”.
Reuses	<code>services/ingestion.py</code> , <code>services/market_feeds.py</code> (<code>feed_status</code>), <code>core/holidays.py</code> , v1 spike guard.
New code	<code>brain/modules/m01_gateway/</code> , <code>brain/connectors/base.py</code> .
Data	Writes <code>quality</code> into <code>feature_snapshots</code> once M02 exists; otherwise only into the run trace.
API and screens	Adds quality to <code>GET /brain/health</code> .
Depends on	M00
Can be built with	M02, M06, M07, M17
Tests	• Stale fixture (last bar 3 days old) → score < 0.6, not fresh. • Split-day fixture with a corporate action → not flagged. • Missing feed → overall issues list names it in plain words.

Done when you see

1. Set M01 to ON: `swingtrade brain set M01 on`.
2. Run a nightly run.
3. Stop the ingest for one symbol (or use `--as-of` a day after the last bar).

Expected: In `runs/latest`, each stock shows a quality score near 1.0. With the stale date, the affected stock shows a low score and its decision says “data is 2 trading days old”.

M02 Perception

Step module

Wave 1

Effort 5

Turns raw prices into the facts the rest of the brain uses: trend, momentum, volatility, volume and market context for each stock. It stores a dated copy every day so any past decision can be replayed exactly.

Brain step	1 Perceive
Covers	BR-13 · NFR-06, NFR-07 · v5: 3.1 Perception (process inputs, clean, feature engineering, pattern detection, signal extraction)
Reads (input)	<code>DataQuality@1</code> (optional), candles via <code>DatedReader</code>
Writes (output)	<code>Snapshot@1</code> per symbol
Processing	1. Call <code>ml/features.py latest_feature_row()</code> with the context frames from <code>ml/market_context.py</code>, the same way <code>ml/predict.py</code> does. 2. Add three convenience fields used across modules: <code>close</code>, <code>atr_14</code>, <code>adv_inr_20</code> (average daily traded value from <code>risk.average_daily_traded_value</code>). 3. Stamp <code>feature_set_version</code> (hash of the feature column list) so models and snapshots can be matched. 4. Store to <code>feature_snapshots</code> (one row per symbol per nightly run).
If switched off	The step fallback computes the same snapshot in memory without storing it. Decisions are unaffected; only replays lose history for those days.
Reuses	<code>ml/features.py</code> , <code>ml/market_context.py</code> , <code>services/risk.py</code> <code>average_daily_traded_value</code> .
New code	<code>brain/modules/m02_perception/</code> .
Data	Migration: <code>feature_snapshots</code> (TimescaleDB hypertable on <code>as_of</code>).
API and screens	None.
Depends on	M00
Can be built with	M01, M06, M07, M17
Tests	• Snapshot equals the feature row the active model sees for the same date. • Row with future candles present → snapshot ignores them. • Version hash changes when a feature is added.

Done when you see

1. `swingtrade brain set M02 on`, then a nightly run.
2. Query: `SELECT symbol, as_of, features->>'rsi_14' FROM feature_snapshots ORDER BY as_of DESC LIMIT 5;`

Expected: One stored snapshot per watchlist stock for today, and the trace line “M02 used, 30 snapshots, 1.8 s”.

M03 State engine

Step module

Wave 2

Effort M

Answers “what is the current state?” in one place: the market, each sector, each stock, your portfolio and the system itself (halted, data OK, broker connected). Plug-ins like the market and sector brains fill in more detail.

Brain step	2 State
Covers	BR-06, BR-14 · v5: 3.2 State engine (market, sector, stock, portfolio, user, system state)
Reads (input)	<code>Snapshot@1</code> , <code>DataQuality@1</code> , positions and cash via <code>DatedReader</code>
Writes (output)	<code>MarketState@1</code> (basic), <code>StockState@1</code> (basic), <code>PortfolioState@1</code> , <code>SystemState@1</code>
Processing	1. Market (basic): <code>classify_regime()</code> on NIFTY → trend and volatility; breadth from <code>load_market_breadth</code>; mode NORMAL, or DEFENSIVE if the trend is down or unknown or volatility elevated. 2. Stock (basic): trend (close vs 50/200-day averages), relative strength vs NIFTY over 20 and 60 days, distance from the 52-week high, from the snapshot. 3. Portfolio: <code>risk.open_holdings()</code>, <code>current_drawdown()</code>, sector exposure, free slots from <code>limits_for()</code>, for the run's book (paper or real). 4. System: <code>system_state.get_state()</code> (halted, exits disabled), overall data quality, Kite session status. 5. Merge rule: plug-ins M10, M11, M12 and M14 may fill fields M03 left empty or refine them; M03's values are the floor.
If switched off	Market trend from <code>classify_regime</code> only; other states empty; unknown trend → DEFENSIVE.
Reuses	<code>m1/market_context.py</code> , <code>services/risk.py</code> , <code>services/limits.py</code> , <code>services/system_state.py</code> , <code>services/deployable.py</code> .
New code	<code>brain/modules/m03_state/</code> .
Data	None new (state is kept in the run trace).
API and screens	State appears in <code>GET /brain/runs/latest</code> under <code>state</code> .
Depends on	M00 (M02 recommended)
Can be built with	M08, M10, M16
Tests	• Down-trend fixture → DEFENSIVE with reason. • Halted system → <code>SystemState.entries_halted</code> and the reason propagate. • Portfolio numbers equal <code>/portfolio</code> for the same book.

Done when you see

1. `swingtrade brain set M03 on`, run nightly.
2. Open `/api/v1/brain/runs/latest` and look at `state`.
3. Halt from the Safety page and run again.

Expected: A state block reading, in words, e.g. “Market: up-trend, normal volatility, 58% of stocks above their 50-day average · Portfolio: 2 positions, 7.4% drawdown · System: running”. After the halt, the banner changes to NO NEW TRADES with “Owner paused new buys”.

M04 Situation recognition

Step module

Wave 3

Effort M

Puts a name on what is going on: “pullback in an up-trend”, “breakout”, “correction”, “crash”, or “never seen this before”. When the market looks abnormal it suggests going defensive. Named situations are what the memory module uses to find similar past cases.

Brain step	3 Recognise
Covers	BR-15 · v5: 3.3 Situation recognition (similar situations, unknown or abnormal, regime detection, anomaly detection, suggest defensive mode) · F26, F35, F36
Reads (input)	MarketState@1, StockState@1, Snapshot@1
Writes (output)	Situation@1 for the market and each stock
Processing	1. Market labels from rules on the state: up-trend, sideways, correction (NIFTY –5% to –10% from its high), bear phase (below –10% and below the 200-day), crash (–4% in a day or VIX jump > 30%), recovery. 2. Stock labels from the snapshot: pullback in up-trend, breakout (close above a 20-day high with volume), base (tight range), breakdown, extended. 3. Unknown detection: distance of today's market state from all past states (standardised features, nearest-neighbour distance); beyond the 99th percentile of historical distances → <code>is_unknown</code>. 4. Suggest defensive when crash, bear phase or unknown; M10 or the constitution turns the suggestion into a mode. 5. Market episodes: when a market label changes, open or close a row in <code>brain_episodes</code> (the start of market memory).
If switched off	Situation “unlabelled”, confidence 0, no defensive suggestion.
Reuses	Snapshot features; <code>classify_regime</code> .
New code	<code>brain/modules/m04_situations/</code> with <code>rules.py</code> (pure) and <code>novelty.py</code> .
Data	Migration: <code>brain_episodes</code> . One-off backfill command labels the last 5 years so memory has history.
API and screens	Situations appear in the run and on cards (“Situation: pullback in an up-trend”).
Depends on	M03
Can be built with	M11, M12, M13
Tests	• March 2020 replay → crash, then bear phase, then recovery. • Label rules on synthetic series (each label has a fixture). • Unknown detector flags a synthetic out-of-range day.

Done when you see

1. `swingtrade brain replay --date 2020-03-23 --modules M03,M04`.
2. Run today's nightly with M04 ON.

Expected: For 23 March 2020 the market situation reads “crash” with “suggest defensive: yes”.

Today's run shows a label for the market and each stock, and `brain_episodes` lists past corrections with start and end dates.

M05 Memory and experience

Step module

Wave 4

Effort L

Remembers what happened after similar situations in the past: how often the +8% target was hit before the −4% stop, how long it took, and the typical path. That evidence is what the decision card shows instead of a promise.

Brain step	4 Remember
Covers	BR-16 · P11 · v5: 3.4 Memory (market, strategy, failure memory, success patterns, lessons, unknown situations, benchmark memory) · F18, F35
Reads (input)	<code>Situation@1</code> , <code>StockState@1</code> , <code>MarketState@1</code> , history via <code>DatedReader</code>
Writes (output)	<code>Recall@1</code> per candidate and holding
Processing	1. Build the experience table once (backfill) and then nightly: for every past stock-day in the universe, its state key (market label, stock label, trend bucket, volatility bucket) and its barrier outcome (+8% before −4% within 15 trading days), plus the daily return path. 2. Lookup: same state key; if fewer than 30 cases, widen by dropping the least important part of the key (volatility, then market label); record which key was used. 3. Return hit rate, median return, 25th–75th percentile band, median days to outcome, and the day-by-day band (used by M15). 4. Only outcomes that were fully known by <code>as_of</code> count (no look-ahead). 5. Also recall the brain's own past decisions in the same situation once enough exist (strategy memory).
If switched off	“No past experience”: M08 shows the model's overall calibration record instead (<code>ml/calibration.py</code>).
Reuses	<code>ml/features.py</code> <code>build_label</code> (barrier label), <code>ml/predict.py</code> <code>barrier_outcome_at_horizon</code> , candles.
New code	<code>brain/modules/m05_memory/</code> , backfill command <code>swingtrade brain memory-build</code> .
Data	Table <code>brain_experience</code> (stock-day, state key, outcome, path) as a hypertable; about 30 stocks × 1,250 days ≈ 40k rows.
API and screens	Recall shown on cards and in <code>GET /brain/why/{symbol}</code> .
Depends on	M04 (for labels; can start with trend buckets only)
Can be built with	M14, M15
Tests	• A case whose outcome resolves after <code>as_of</code> is excluded. • Widening happens below 30 cases and is recorded. • Hit rate on a synthetic series with a known answer.

Done when you see

1. `swingtrade brain memory-build` (takes a few minutes).
2. `swingtrade brain why <SYMBOL>`.

Expected: The “why” output includes a sentence like “Similar cases: 124 · reached +8% first in 38% · middle half ended between −3% and +5% · median 9 trading days”.

M06 Reasoning and models

Step module

Wave 1

Effort M

Collects every opinion about each stock in one common form: the ML model's probability, each strategy's view, and plug-in views, each with a confidence and reasons. It also knows how reliable each source has been, and resolves conflicts between them.

Brain step	5 Reason
Covers	BR-17, BR-35 · v5: 3.5 Reasoning (rule engine, ML ensemble, statistical analysis, multi-factor scoring, historical similarity, conflict resolution, uncertainty) · Strategy brain · F04, F25
Reads (input)	Snapshot@1, states, Situation@1, Recall@1 (all optional)
Writes (output)	Opinion@1 (many per symbol) and one combined opinion per symbol
Processing	1. Model opinion: active model via <code>ml/predict.py</code> <code>predict_instrument()</code>; stance = $2p - 1$; confidence from the calibration bucket. 2. Strategy opinions: call each active v1 strategy's pure <code>evaluate()</code> (ML swing, SMA crossover); BUY → +, HOLD → 0; reasons copied. 3. Plug-in opinions arrive from M10–M13 in the same step. 4. Combine: weighted average of stances; weight = source's measured reliability by regime (from signal outcomes), default equal weights when no record. 5. Conflict rule: if strong sources disagree (one > +0.3, one < -0.3), the combined confidence is halved and the reason says so. 6. Uncertainty: combined confidence below 0.3 → the opinion is marked uncertain, which M08 turns into WAIT.
If switched off	Active ML model probability only. No active model → no opinion → WAIT.
Reuses	<code>ml/predict.py</code> , <code>ml/registry.py</code> , <code>ml/calibration.py</code> , <code>strategies/*</code> , signal outcome columns.
New code	<code>brain/modules/m06_reasoning/</code> .
Data	None new.
API and screens	Opinions listed in the trace and in “why”.
Depends on	M00
Can be built with	M01, M02, M07, M17
Tests	• Model opinion equals <code>predict_instrument</code> for the same date. • Conflict fixture halves confidence. • No active model → empty opinions, no crash.

Done when you see

1. `swingtrade brain set M06 on`, then `swingtrade brain why <SYMBOL>`.

Expected: “Opinions: model 0.61 (calibrated: fair), ML swing: buy, SMA crossover: no view · combined +0.32, confidence 0.55”.

M07 Risk and safety gate

Mandatory

Wave 1

Effort M

The brain's non-negotiable safety check. For every possible trade it asks the existing risk rules whether it is allowed and how big it may be, and it applies the market mode. Without this module the brain will never suggest a new trade.

Brain step	6 Risk
Covers	BR-04, BR-18, BR-47 · C1–C5 · v5: 3.6 Risk and safety, Risk brain (exposure), Constitution: risk limits, uncertainty handling, human override
Reads (input)	Opinion@1, PortfolioState@1, SystemState@1, MarketState@1, DataQuality@1, Snapshot@1
Writes (output)	RiskVerdict@1 per candidate
Processing	1. For each symbol with a positive combined opinion: stop = entry × 0.96 (locked rule) and call <code>risk.check_entry()</code> with the run's book, portfolio value and cash. The call is read-only by design. 2. Map the result: allowed, <code>quantity</code> → <code>max_qty</code>, <code>rule</code>, plain <code>reason</code>, <code>amount_inr</code>. 3. Apply the market mode: DEFENSIVE multiplies <code>max_qty</code> by the configured factor (default 0.5) and allows only the top N by confidence (default 2); NO NEW TRADES rejects all with the mode's reason. 4. Data quality below the floor → reject with “data not reliable today”. 5. Holdings: pass through v1 exit state (stop hit, target hit) so M08 never suggests HOLD on something v1 is exiting. 6. Manifest <code>mandatory=True</code>; the registry refuses mode OFF.
If switched off	None. If M07 fails, the constitution sets NO NEW TRADES for the run.
Reuses	<code>services/risk.py</code> (<code>check_entry</code> , <code>calculate_quantity</code> , <code>open_holdings</code>), <code>services/limits.py</code> , <code>services/avoid.py</code> , <code>services/system_state.py</code> , <code>services/deployable.py</code> .
New code	<code>brain/modules/m07_risk/</code> .
Data	None new. Rejections are in the run trace (v1 <code>risk_events</code> stays v1-only).
API and screens	Verdicts shown in “why”; console blocks OFF for M07.
Depends on	M00
Can be built with	M01, M02, M06, M17
Tests	• Verdict equals <code>check_entry</code> for the same inputs (adapter test). • DEFENSIVE halves size and limits count. • PUT mode OFF for M07 → 409 with a plain reason. • Exception inside <code>check_entry</code> → run completes, banner NO NEW TRADES.

Done when you see

1. `swingtrade brain set M07 on` and run nightly.
2. Try `swingtrade brain set M07 off`.
3. Halt from the Safety page, then run again.

Expected: The banner changes from NO NEW TRADES to NORMAL (or DEFENSIVE), stocks with strong opinions show a size, and rejected ones say why (“Sector already at 25% of the account”). Switching M07 off is refused. After the halt, all ideas are WATCH with “Owner paused new buys”.

M08 Decision engine

Step module

Wave 2

Effort M

Turns opinions, evidence and risk verdicts into the final word for each stock (TRADE, WATCH, WAIT, AVOID) and each holding (HOLD, MONITOR, REDUCE, EXIT), with levels, size and an honest evidence sentence. It also sets the market banner.

Brain step	7 Decide
Covers	BR-19, BR-41 · P9, P11 · v5: 3.7 Decision engine (confidence score, expected return range, duration, action rationale, opportunity cost), Decision states
Reads (input)	everything from steps 1–6
Writes (output)	<code>Decision@1</code> , <code>Banner@1</code>
Processing	1. Ideas: AVOID if a restriction, event blackout or breakdown applies; TRADE if combined stance $\geq$ threshold, not uncertain, verdict allowed and quality fresh; WATCH if the opinion is good but something blocks it (the blocker becomes the reason); otherwise WAIT. 2. Levels: entry zone = close $\pm$ 0.25 $\times$ ATR (capped at $\pm$1%), target = entry mid $\times$ 1.08, stop = entry mid $\times$ 0.96, horizon 15. 3. Evidence text from <code>Recall@1</code> when $n \geq 30$, else from the calibration record; always with the sample size. 4. Holdings: EXIT if v1 is exiting or the stop was hit (reported as “Stop hit”); REDUCE at the first target (half) or in DEFENSIVE with an oversized position; MONITOR on drift; else HOLD. 5. Opportunity cost: when slots are full, compare the best TRADE with the weakest holding; if clearly better, add “Consider replacing X” to the holding's reasons (never an automatic swap). 6. Ranking: order TRADEs by confidence $\times$ evidence; M14 may re-rank. 7. Banner: NO NEW TRADES > DEFENSIVE > NORMAL, with reasons collected from state, situations and M07.
If switched off	Threshold map: model probability $\geq$ the strategy's threshold and verdict allowed $\rightarrow$ TRADE; else WAIT; holdings $\rightarrow$ HOLD. No ranges, no opportunity cost.
Reuses	<code>risk.rank_buy_candidates</code> , <code>exit_policy.py</code> (first target, trailing), strategy thresholds.
New code	<code>brain/modules/m08_decide/</code> with <code>words.py</code> , <code>levels.py</code> , <code>evidence.py</code> .
Data	Writes <code>brain_decisions</code> through the runner.
API and screens	<code>GET /brain/decisions</code> , <code>GET /brain/banner</code> .
Depends on	M00, M07 (M06 recommended)
Can be built with	M03, M10, M16
Tests	• Word table: every combination of opinion, verdict, quality and mode $\rightarrow$ expected word. • Stop always 4% below entry mid; target 8% above. • No text contains “will”, “guaranteed” or “sure”. • Stop-hit holding $\rightarrow$ EXIT with “Stop hit”.

Done when you see

1. Turn M06, M07, M08 on; run nightly.
2. Open `/api/v1/brain/decisions?word=TRADE` .

Expected: A short list of TRADE ideas with entry zone, target (+8%), stop (–4%), size and an evidence sentence including the number of cases; every other stock has WATCH, WAIT or AVOID with a one-line reason.

M09 Learning loop

Step module

Wave 5

Effort L

After trades finish, compares what the brain expected with what actually happened, spots when the models are drifting, investigates failures, and writes proposals for improvements. It never changes anything by itself: the owner accepts or rejects each proposal.

Brain step	8 Learn
Covers	BR-20 · P12, C9 · v5: 3.8 Learning (track outcomes, expected vs actual, drift, failure investigation, update experience, improve models, adapt rules, replay and counterfactuals), 6 Feedback loop · F18, F23, F31, F39
Reads (input)	<code>brain_decisions</code> , outcomes, <code>brain_track</code> , snapshots
Writes (output)	<code>Outcome@1</code> , <code>Proposal@1</code>
Processing	1. Outcomes: for each resolved decision (target, stop, time), record the return, days and maximum moves, reusing <code>evaluate_pending_signals</code> logic. 2. Expected vs actual: hit rate per confidence band vs the evidence the card showed; weekly report. 3. Drift: population stability of key features (snapshot vs training window) and outcome drift by band; alert above thresholds. 4. Failure investigation: for each stop-out, which situation, which opinions, what the market did; group into patterns. 5. Counterfactuals: replay the week with each module OFF (from stored snapshots) to measure each module's contribution. 6. Proposals: e.g. "raise TRADE threshold to 0.60 (would have avoided 4 of 6 losses, cost 1 of 5 wins)", "switch M12 from SHADOW to ON", "train a challenger model". Stored as open.
If switched off	Nothing proposed. v1's evaluation jobs keep recording outcomes, so no data is lost.
Reuses	<code>ml/predict.py</code> evaluation functions, <code>ml/calibration.py</code> , <code>ml/train.py</code> (challenger only, never activated automatically).
New code	<code>brain/modules/m09_learn/</code> , job <code>brain_learn</code> , endpoints <code>/brain/proposals</code> .
Data	Migration: <code>brain_proposals</code> ; outcome columns on <code>brain_decisions</code> .
API and screens	Proposals list with Accept / Reject in the console.
Depends on	M08 and about 4 weeks of stored decisions
Can be built with	None in its wave (M18 follows it)
Tests	• Proposal never changes config until accepted. • Counterfactual replay with a module off reproduces stored decisions for the on case. • Drift alarm on a shifted synthetic feature.

Done when you see

1. Run `swingtrade brain learn --since 2026-10-01` on stored runs (or a replayed month).
2. Open the console's Proposals tab.

Expected: A weekly "expected vs actual" table by confidence band and at least one proposal with its evidence and Accept / Reject buttons. Nothing changes until you press Accept.

M10 Market brain

Plug-in

Wave 2

Effort M

A specialist for the whole market. It reads trend, breadth, volatility, India VIX and foreign/domestic fund flows, and decides the market mode: NORMAL, DEFENSIVE or NO NEW TRADES. This is the brain's bear-market protection.

Brain step	2 State (+ opinions)
Covers	BR-30 · v5: Market brain, 3.2 market state, NO-TRADE logic, defensive mode · A1, F01, F26, F36
Reads (input)	<code>MarketState@1</code> (basic), index/VIX/breadth, FII/DII flows
Writes (output)	<code>MarketState@1</code> (refined: mode, reasons), market <code>Opinion@1</code>
Processing	- Score four signals, each -1..+1: trend (50/200-day and slope), breadth (% of stocks above 50-day), volatility (VIX level and 5-day change), flows (FII net 5 and 20 days). - Mode rules: NO NEW TRADES if drawdown halt, crash situation or quality floor breached; DEFENSIVE if the score sum < 0 or the situation says defensive; else NORMAL. Hysteresis: two consecutive runs are needed to leave DEFENSIVE. - Plain reasons: “More than 60% of stocks are below their 50-day average”, “FIIs sold ₹12,400 crore in 5 days”. - Market opinion applied to all stocks as a small tilt (± 0.1).
If switched off	M03's basic market state; mode from trend and drawdown only.
Reuses	<code>market_context.load_index_candles</code> , <code>load_vix_candles</code> , <code>load_market_breadth</code> , <code>market_feeds.load_fii_dii_data</code> , <code>deployable.py</code> .
New code	<code>brain/modules/m10_market/</code> .
Data	None new; mode changes also logged to <code>brain_episodes</code> once M04 exists.
API and screens	Banner reasons.
Depends on	M03
Can be built with	M08, M16
Tests	- Replay 2020-03 and 2022-06 → DEFENSIVE or NO NEW TRADES on the known bad weeks. - Hysteresis prevents flip-flopping on a noisy fixture.

Done when you see

- Turn M10 on; run nightly.
- `swingtrade brain replay --date 2022-06-17 --modules M03,M10`.

Expected: Today's banner shows a mode with 2–4 plain reasons. The June 2022 replay shows DEFENSIVE with reasons about breadth and FII selling.

M11 Sector brain

Plug-in

Wave 3

Effort 5

Ranks sectors by strength and spots rotation: which sectors are leading, improving, weakening or lagging. Strong sectors get a small boost and help break ties between similar ideas.

Brain step	2 State (+ opinions)
Covers	BR-31 · v5: Sector brain (sector rotation), 3.2 sector state · F02, F46
Reads (input)	Sector index candles, <code>StockState@1</code>
Writes (output)	<code>SectorState@1</code> , small sector <code>Opinion@1</code> tilt
Processing	1. Relative strength of each sector index vs NIFTY over 20 and 60 days; rank. 2. Rotation quadrant from 20-day vs 60-day relative strength (leading, improving, weakening, lagging). 3. Map each stock to its sector with <code>m1/sector_map.py</code>. 4. Tilt: leading +0.1, improving +0.05, weakening -0.05, lagging -0.1.
If switched off	No sector ranking; no tilt. The 25% sector cap still applies inside M07.
Reuses	<code>m1/sector_map.py</code> , <code>market_context.load_sector_candles</code> .
New code	<code>brain/modules/m11_sector/</code> .
Data	None new.
API and screens	Sector rank on cards (“Sector: IT, ranked 3 of 14, improving”).
Depends on	M03
Can be built with	M04, M12, M13
Tests	• Ranking on synthetic sector series. • Every watchlist stock maps to a sector (or “unknown”).

Done when you see

1. Turn M11 on; `swingtrade brain why <SYMBOL>`.

Expected: “Sector: Banks · rank 2 of 14 · leading” in the why output, and a sector table in the run.

M12 Stock brain

Plug-in

Wave 3

Effort M

A specialist for each stock. It spots classic swing setups (pullback, breakout, base), reads delivery percentage and big-investor deals, and builds a short profile of how the stock usually behaves.

Brain step	3 Recognise (+ opinions)
Covers	BR-32 · v5: Stock brain, 3.1 pattern detection · F03, F07, F40, F42
Reads (input)	Snapshot@1, StockState@1, delivery, bulk and block deals
Writes (output)	stock Situation@1 (setup), StockState@1 refinements, stock Opinion@1
Processing	1. Setups (exact rules, each one tested in replay): pullback to the 20-day average in an up-trend; breakout above a 20-day high on 1.5× volume; tight base (10-day range < 1.5 × ATR). 2. Delivery signal: delivery % above its 20-day average for 3 of the last 5 days. 3. Deals: any bulk or block buy by a known institution in the last 10 days; sells count negatively. 4. Profile: typical daily range, gap frequency, average days to +8% historically. 5. Opinion: +0.2 per confirming signal (max +0.4), −0.2 on breakdown or institutional selling.
If switched off	No setup labels or stock opinion. Cards use the model reasons.
Reuses	market_feeds tables (bhavcopy delivery, deals), m1/features.py indicators.
New code	brain/modules/m12_stock/ with setups.py (pure).
Data	None new.
API and screens	Setup and deal lines on cards.
Depends on	M02, M03
Can be built with	M04, M11, M13
Tests	• One fixture per setup; negative fixtures that must not match. • Deal parsing on a stored sample.

Done when you see

1. Turn M12 on (start in SHADOW, then ON); run nightly and open a why.

Expected: Cards show lines like “Setup: pullback to the 20-day average in an up-trend” and “Delivery above average 4 of the last 5 days”. In SHADOW, the same lines appear only in the trace.

M13 News and events brain

Plug-in

Wave 3

Effort M

Knows the calendar: results dates, dividend ex-dates, splits and exchange restrictions. It keeps the brain away from stocks just before results, and marks stocks with fresh events. It uses the official exchange data we already collect, not web scraping.

Brain step	3 Recognise (+ risk windows)
Covers	BR-33 · v5: News brain (event analysis), 1 News and events, 3.6 event risk · F08, F09, F11
Reads (input)	Events, corporate actions and ASM/GSM tables
Writes (output)	stock <code>Situation@1</code> (events), <code>StockState@1.restrictions</code> , negative <code>Opinion@1</code> near events
Processing	1. Results blackout: results within the next 5 trading days → situation “results soon”; M08 makes it AVOID for new ideas and adds a note to holdings. 2. Ex-date and split handling: adjust expectations; never treat an ex-date drop as a breakdown. 3. Restrictions: ASM/GSM membership → AVOID with the list name. 4. Fresh filings (when available from NSE announcements already stored) listed as context, never as a buy reason by themselves. 5. Later (separate proposal): a licensed news feed classified by a language model, with citations.
If switched off	Only v1's avoid list protects around events.
Reuses	<code>market_feeds.load_upcoming_events</code> , <code>load_restrictions</code> , <code>services/avoid.py</code> .
New code	<code>brain/modules/m13_news/</code> .
Data	None new.
API and screens	“Results on 14 Oct” line on cards and in the tracker.
Depends on	M01 (feed freshness), M03
Can be built with	M04, M11, M12
Tests	• A stock with results in 3 days → AVOID for new ideas. • Ex-date day is not a breakdown.

Done when you see

1. Turn M13 on; pick a stock with an upcoming results date; run why.

Expected: That stock shows AVOID with “Results on <date>: the brain avoids new trades 5 trading days before results”.

M14 Portfolio brain

Plug-in

Wave 4

Effort M

Looks at all your positions together instead of one stock at a time: how concentrated you are, which holdings move together, and whether a new idea is better than the weakest thing you already own. It replaces the two portfolio boxes in the picture with one.

Brain step	2 State + 7 Decide
Covers	BR-34 · v5: Portfolio (optimisation) + Portfolio brain (merged), 3.7 opportunity cost · F12, F13, F14, F15, F33
Reads (input)	<code>PortfolioState@1</code> , decisions in progress, candles
Writes (output)	<code>PortfolioState@1</code> refinements (correlation, concentration), ranking and opportunity-cost notes
Processing	1. Correlation of 60-day returns between holdings and each TRADE candidate; flag candidates with correlation > 0.7 to an existing holding. 2. Concentration: sector and single-stock share against the limits in <code>limits_for()</code>. 3. Re-rank TRADEs: prefer candidates that diversify; demote highly correlated ones (reason given). 4. Opportunity cost: compare the best TRADE's evidence with the weakest holding's current outlook; suggest replacement in words only. 5. What-if: <code>POST /brain/whatif</code> shows exposure after a planned trade.
If switched off	Ranking by <code>rank_buy_candidates</code> ; no correlation checks (sector cap still enforced by M07).
Reuses	<code>services/risk.py</code> , <code>services/limits.py</code> , <code>services/portfolio.py</code> .
New code	<code>brain/modules/m14_portfolio/</code> .
Data	None new.
API and screens	Portfolio panel in the console; what-if endpoint.
Depends on	M03, M08
Can be built with	M05, M15
Tests	• Two perfectly correlated fixtures → second is demoted. • Opportunity-cost note appears only when slots are full.

Done when you see

1. Turn M14 on with 2+ paper positions; run nightly.

Expected: TRADE ideas reordered with notes such as “Moves closely with HDFCBANK, which you already hold”, and when slots are full, a holding card says “A stronger idea is waiting: consider replacing”.

M15 Trade tracker and exit intelligence

Step module

Wave 4

Effort M

Follows each open trade day by day and compares it with the typical path of similar trades. It labels the trade on track, drifting or breaking down, explains unexpected moves, and at the target shows the evidence for booking half and trailing the rest.

Brain step	Intraday · 7 Decide (holdings)
Covers	BR-42, BR-43, BR-44 · C6, C7 · v5: 5.2, 5.3, 5.4, 5.5, 4 Day-by-day tracking · F12, F13, F50, F51, F52
Reads (input)	Open positions, <code>Recall@1.typical_path</code> , quotes, <code>MarketState@1</code> , <code>SectorState@1</code>
Writes (output)	<code>TrackPoint@1</code> , holding <code>Decision@1</code> inputs
Processing	1. Band: the 25th–75th percentile path of similar trades from M05 (or the overall path of all past brain TRADEs when memory is off). 2. Status: on track inside the band; drift below the band but above the stop; breakdown below the band's lower bound minus 1 ATR or on a failed setup. 3. Explain drift: list what changed (market mode, sector rank, volume on down days, events) in plain words. 4. Stop hit: if v1 has exited, show “Stop hit: exited at ₹X”; never offer a choice (C7). 5. At the first target (+5%, where the existing exit policy books half): show how much further similar trades went on towards +8% (median and range), to support trailing the rest. 6. Stops only move up (C6).
If switched off	No day-by-day status; holdings show HOLD unless v1 exits act.
Reuses	<code>services/exit_policy.py</code> , <code>job_refresh_quotes</code> prices, v1 positions.
New code	<code>brain/modules/m15_tracker/</code> , frontend <code>HoldingTracker</code> component.
Data	Migration: <code>brain_track</code> (hypertable).
API and screens	<code>GET /brain/track/{symbol}</code> ; chart on the holding page.
Depends on	M08 (M05 for real bands)
Can be built with	M05, M14
Tests	• Price path fixtures for each status. • Stop-hit fixture shows “Stop hit”. • Stop never decreases across a series.

Done when you see

1. Open a paper position (or use a fixture), turn M15 on, let two intraday runs pass (or `swingtrade brain run --kind intraday`).
2. Open the holding page.

Expected: A small chart with the band and the price line, a day counter (“day 3 of up to 15”), and a status word with a one-line reason.

M16 Outputs, cards and alerts

Step module

Wave 2

Effort M

Shows the brain's results where you will see them: the market banner on every page, decision cards for ideas and holdings, and Telegram messages when something changes. The buttons say “Plan this trade” and “Add to watchlist”, never “Invest today”.

Brain step	After the run
Covers	BR-40, BR-41, BR-45 · C12 · v5: 4 Outputs (stock signals, alerts, reports, interfaces), 5.1, 5.2
Reads (input)	<code>brain_decisions</code> , <code>Banner@1</code> , previous run for changes
Writes (output)	Screens and Telegram messages (no brain data)
Processing	1. Banner component under the top bar, from <code>GET /brain/banner</code>. 2. Decision card component: word pill, reason, levels, size, evidence sentence, “what could go wrong”, “why?” link. 3. Ideas page: TRADE first, then WATCH; WAIT and AVOID collapsed. 4. Alerts after each run: banner changes, new TRADE, holding moves to MONITOR, REDUCE or EXIT, via the existing Telegram sender; de-duplicated per decision. 5. Plan gating (when plans are on) decides who sees which cards; the brain is plan-agnostic.
If switched off	Decisions visible only in the console and API.
Reuses	<code>notifications/telegram.py</code> , existing React components, <code>core/plans.py</code> gating.
New code	Frontend: <code>MarketBanner</code> , <code>DecisionCard</code> , page <code>BrainIdeas.tsx</code> ; backend job <code>brain_alerts</code> .
Data	Alert sent-at column on <code>brain_decisions</code> .
API and screens	<code>GET /brain/decisions</code> , <code>GET /brain/decisions/{id}</code> , <code>GET /brain/banner</code> .
Depends on	M08
Can be built with	M03, M10
Tests	• Card text lint (no promise words). • Alert de-duplication. • Banner renders all three modes.

Done when you see

1. Start the front end; open `http://localhost:5173/brain/ideas`.

Expected: A banner at the top, TRADE cards with levels and evidence, and one Telegram test message when the banner changes (with the bot token set locally).

M17 Brain console and observability

Operations

Wave 1

Effort M

Your control room for the brain. It lists every module with an on/shadow/off switch and a health light, shows the last runs and how long they took, lets you run the brain now, and shows a full trace of why any decision was made.

Brain step	Operations
Covers	BR-11, BR-46, BR-47 · C8 · v5: 8 Observability (system health, data freshness, incident detection, audit logs, access control), Human override, Audit logs
Reads (input)	<code>brain_modules</code> , <code>brain_runs</code> , trace, health
Writes (output)	Module modes, overrules, halts (through existing <code>system_state</code>)
Processing	1. Page <code>/brain</code> (owner only): module table with switch, health, version, last status; M07's switch is locked ON. 2. Runs list with timing, counts per word, banner; “Run now” and “Why this stock?” actions. 3. Trace viewer: per decision, the timeline of modules (used, shadow, fallback, skipped) and reasons. 4. Override a decision (more cautious only) with a reason. 5. Health: data freshness from M01, last successful run, failed runs in 7 days; Telegram alert when a nightly run fails twice. 6. Housekeeping job for retention (section 13.1).
If switched off	Modes come from <code>brain.yaml</code> ; no switching without restart.
Reuses	Owner auth dependency, existing Safety page patterns, <code>workers/heartbeat.py</code> .
New code	Frontend page <code>Brain.tsx</code> , endpoints for trace, override and health.
Data	None new (uses M00 tables).
API and screens	Section 16.
Depends on	M00
Can be built with	M01, M02, M06, M07
Tests	• Non-owner gets 403. • Override to a bolder word is refused. • Trace view matches stored events.

Done when you see

1. Open `http://localhost:5173/brain`, switch M05 to OFF, press Run now, open a decision's trace.

Expected: The module list with switches; after the run, the trace for a stock shows “M05 skipped: switched off → Remember used fallback: no past experience”.

M18 Shadow run and go-live switch

Core

Wave 5

Effort M

Connects the brain to real trading, carefully. First the brain runs as a shadow strategy next to v1 and both are scored. Then, with your approval, its ideas need your manual OK before an order. Only after that can it trade automatically.

Brain step	Connection to v1 execution
Covers	BR-48 · P6, P7, C10 · v5: 9 A/B testing (shadow mode), 10 Gradual rollout, Broker execution (owner only)
Reads (input)	<code>brain_decisions</code> , v1 signal outcomes
Writes (output)	v1 <code>signals</code> rows through the normal strategy path
Processing	<code>strategies/brain.py</code>: <code>@register_strategy</code> class <code>BrainStrategy</code>, <code>strategy_type="brain"</code>; <code>evaluate()</code> reads today's stored brain decision and returns a <code>SignalDecision</code> (TRADE → BUY with stop, target, horizon; others → HOLD with the reason). - Create the strategy row as a shadow strategy (<code>is_shadow</code> signals), so v1 execution, risk and exits score it without trading. - Comparison report: brain vs v1 on the same days (hit rate, return after costs, drawdown, number of trades). - Stage 2, manual approval: brain TRADEs create a pending approval (Telegram + console); an order goes through v1 execution only after the owner taps Approve. - Stage 3, automatic: owner flips the strategy to active. Rollback is one switch back to shadow (and v1 tag stays available). - This is the only module allowed to touch the execution path, and only through the strategy interface.
If switched off	Without M18 the brain is advisory only; v1 trades as today.
Reuses	<code>strategies/base.py</code> , <code>services/engine.py</code> , <code>services/execution.py</code> , signal outcome columns, avoid and risk services.
New code	<code>strategies/brain.py</code> , approvals endpoint and console tab, comparison report.
Data	No new tables; uses <code>signals</code> (<code>strategy_type="brain"</code>).
API and screens	<code>GET /brain/compare</code> , <code>POST /brain/approvals/{id}</code> .
Depends on	M08, M16, M17 and at least 30 resolved shadow decisions for stage 2
Can be built with	None (last)
Tests	- BrainStrategy returns the same word as the stored decision. - Shadow signals never create orders. - Approval required before any order in stage 2.

Done when you see

- Create the brain strategy in shadow locally; run the v1 scan (`swingtrade scan`).
- Open Strategies → Brain.

Expected: The Brain strategy appears in the Strategies page with shadow signals scored like the others, and the comparison table shows brain vs v1 side by side. No orders are placed.

M19 User brain

Later

Wave Later

Effort M

Adapts what each user sees to their own settings: capital, risk level and watchlist. For now this means only personal views and position sizing. Personal advice waits for a legal review.

Brain step	Inputs + 7 Decide
Covers	BR-07, BR-36 · v5: User brain (personalisation), Personalised recommendations · F19
Reads (input)	User settings, <code>Decision@1</code>
Writes (output)	Per-user size and view filters (not new decisions)
Processing	1. Size each TRADE for the user's capital using <code>limits_for(user_capital)</code>. 2. Filter by the user's watchlist and risk level (e.g. hide small caps for “low” risk). 3. Never change the decision word per user; the brain's word is the same for everyone.
If switched off	Everyone sees the owner-scale decisions without personal sizing.
Reuses	<code>services/limits.py</code> , plans and user tables.
New code	<code>brain/modules/m19_user/</code> .
Data	User preference columns (with the plans work).
API and screens	<code>GET /brain/decisions?for=me</code> .
Depends on	M14, the plans branch, legal review
Can be built with	—
Tests	• Same decision word for every user. • Size scales with capital.

Done when you see

1. Create a second local user with ₹50,000 capital; open ideas as that user.

Expected: Same TRADE ideas, sized for ₹50,000, with small caps hidden for a low-risk profile.

17 • Order of work: serial and parallel

Only one module must be built on its own first: **M00, the core**. After that, work runs in waves. Modules inside a wave do not depend on each other and can be built at the same time; each wave needs the one before it.

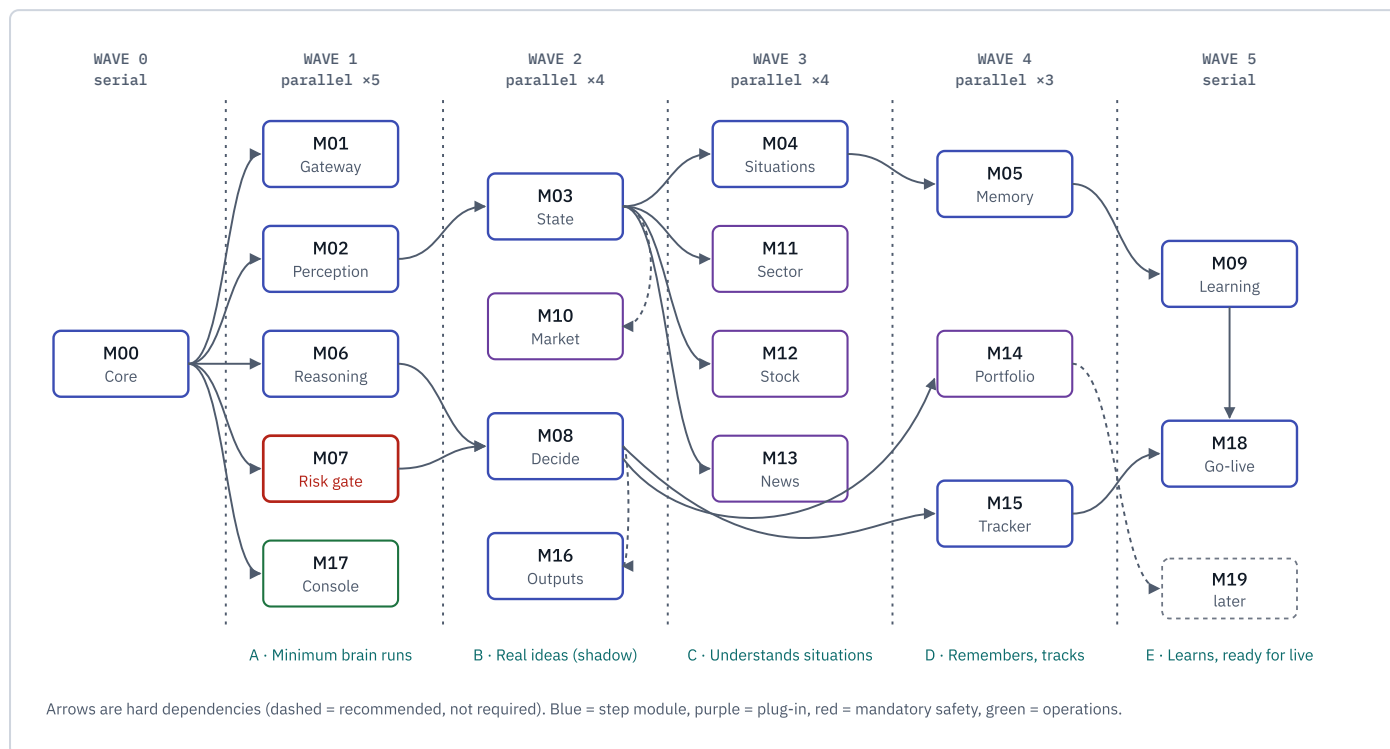


Figure 8. Build waves and dependencies. Critical path: M00 → M02 → M03 → M04 → M05 → M09 → M18.

WAVE	MODULES	SERIAL OR PARALLEL	MILESTONE YOU CAN CHECK	EFFORT (SESSIONS)
0	M00	Serial, alone	An empty but safe brain runs: all WATCH/WAIT, NO NEW TRADES	≈ 4–5
1	M01, M02, M06, M07, M17	Parallel (5 independent)	A: minimum brain with real model opinions, risk verdicts, quality scores and the console	≈ 2–3 each
2	M03, M08, M10, M16	Parallel; M10 after M03 is merged if done by one builder	B: real TRADE ideas with levels, a market banner and cards, in shadow	≈ 2–3 each
3	M04, M11, M12, M13	Parallel	C: situations, sector rank, setups, event windows	≈ 1–3 each
4	M05, M14, M15	Parallel	D: evidence from similar cases, portfolio view, day-by-day tracking	≈ 2–5 each
5	M09 → M18	Serial (M18 needs M09's comparison data and 30 resolved decisions)	E: learning proposals; shadow vs v1 comparison; manual-approval stage	≈ 4 + 3
Later	M19	After the plans branch and a legal review	Per-user sizing	≈ 2–3

17.1 What “parallel” means in practice

- **One builder (one Claude session at a time):** build the wave's modules one after another in any order. Parallel just means the order inside a wave doesn't matter.
- **Several builders at once:** each module gets its own branch and git worktree (`brain/m06-reasoning`). They only share `brain/contracts.py` , which M00 fixed. A module that needs a contract change stops and asks.
- **Merging:** each module merges into a `brain` integration branch after its tests and local check pass; the integration branch is rebased on `main` regularly. Nothing reaches production until the owner merges to `main` , and even then `BRAIN_ENABLED=false` keeps it idle.

17.2 Where the existing work fits

EXISTING ITEM	RELATION TO THE BRAIN	RECOMMENDATION
Live bug: orders after market close (roadmap tracker)	v1 execution, not brain	Fix before or alongside wave 0; it affects real trading today
Barrier models not on production	M06 reads whatever model is active	Deploy barrier models in shadow now; M06 benefits automatically
v2 profiles and factor branch	v2 profiles become inputs to M06 and M10 later	Keep separate; the brain does not block it
Plans branch (Free / Plus / Pro)	Decides who sees brain cards (M16) and M19	Independent; merge when reviewed

PART E · BUILD PLAN

18 · How to ask for a module

A short, fixed request works best. Copy this and change the module number:

Take module M06 (Reasoning and models) from the TradeMind Brain Build Book and finish it. Follow its module sheet and sections 10–12. Start it in SHADOW, run its tests, then start the local server so I can check it.

18.1 What Claude does for every module

1. **Checks readiness:** are the module's dependencies merged? If not, says which, and offers to build them first.
2. **Creates a branch** `brain/m06-reasoning` from the `brain` integration branch.
3. **Writes the tests first** from the sheet's Tests row, plus the three contract tests every module must pass (section 20).
4. **Builds the module** inside `brain/modules/m06_reasoning/`, reusing the listed code, adding its migration if it has one.
5. **Registers it** with `default_mode=SHADOW`.
6. **Runs all brain tests** (not just the module's) with Docker Postgres up.
7. **Starts the local server** (section 19) and performs the sheet's “Done when you see” steps itself first.
8. **Reports** in plain words: what was built, what to click, what you should see, test results, anything that did not go as planned.
9. **Waits for you** to check, then switches it ON and merges into `brain` when you say so.

18.2 Definition of done (every module)

#	CHECK
D1	Manifest declares id, step, kind, version, reads, writes, budget.
D2	Module passes the shared contract tests: valid output, honours OFF, survives its own exception.
D3	The sheet's own tests pass; all brain tests still pass.
D4	The monotonicity test still passes with this module ON and OFF.
D5	No import from <code>brokers</code> or <code>services.execution</code> (except M18's strategy file).
D6	Every reason string is plain English (NFR-14).
D7	The “Done when you see” check passes on the local server.
D8	The console shows the module with its health and mode.
D9	The traceability table (section 23) row is updated to “built”.

18.3 Other useful requests

- “Build wave 1.” Claude builds M01, M02, M06, M07 and M17 one after another (or in parallel worktrees if you ask), with one local check at the end.
- “Switch M12 from shadow to on.” Claude shows the shadow record first, then switches it.
- “Replay last month with M05 off.” A counterfactual run; shows what changed.
- “Why did the brain say WATCH for RELIANCE?” Claude reads the trace and explains it in plain words.

PART E · BUILD PLAN

19 · Local server runbook

Everything runs on your Windows PC. Only Postgres and Redis run in Docker; the API and front end run directly so changes reload instantly.

19.1 Start

```
# 1. Database and cache (Docker Desktop must be running)
docker compose up -d postgres redis

# 2. Check Postgres really answers (a port check is not enough)
docker compose exec postgres pg_isready -U swingtrade

# 3. Backend: migrations, then the API with auto-reload
cd backend
.\.venv\Scripts\Activate.ps1
alembic upgrade head
$env:BRAIN_ENABLED = "true"
uvicorn swing_trade_ml.main:app --reload           # http://localhost:8000/docs

# 4. Front end (second terminal)
cd frontend
npm run dev                                       # http://localhost:5173
```

19.2 Brain commands

```
swingtrade brain modules           # list modules, modes, health
swingtrade brain set M06 on|shadow|off  # switch a module
swingtrade brain run --kind nightly     # run now
swingtrade brain run --kind nightly --as-of 2026-09-15
swingtrade brain why RELIANCE         # one stock, full reasoning in words
swingtrade brain replay --date 2022-06-17 --modules M03,M10
swingtrade brain memory-build         # M05 backfill
swingtrade brain learn --since 2026-10-01  # M09
```

19.3 Where to look

WHAT	WHERE
Brain console (modules, runs, traces)	<code>http://localhost:5173/brain</code>
Ideas and cards	<code>http://localhost:5173/brain/ideas</code>
API explorer	<code>http://localhost:8000/docs</code> → “brain”
Latest run as JSON	<code>http://localhost:8000/api/v1/brain/runs/latest</code>

19.4 Common problems

YOU SEE	CAUSE	FIX
Tests or the API hang for minutes, then fail	Docker Postgres is not running	Start Docker Desktop; <code>docker compose up -d postgres redis</code> ; check with <code>pg_isready</code>

All decisions WATCH, banner NO NEW TRADES	M07 not built or failing	Expected until M07 is done; otherwise open the trace
All decisions capped at WATCH, “data not checked”	M01 off	Switch M01 on or build it
No candles for today	Local database has no fresh data (Kite logins go to production)	Use <code>--as-of</code> with the last stored date, or run <code>swingtrade backfill</code>
Port 5432 in use	A local PostgreSQL install	Docker publishes on 5433 already; check <code>.env</code>

PART E • BUILD PLAN

20 • Testing

Four layers of tests. The first two run on every module; the last two protect the whole brain.

LAYER	WHAT IT PROVES	WHERE	RUNS
1 • Unit	The module's pure logic is right (setups, labels, levels, scores)	<code>brain/modules/mXX_*/tests/</code>	Every commit
2 • Contract	The module's output matches its declared contracts; it honours OFF; an exception inside it does not break the run	<code>brain/tests/test_contracts.py</code> , parametrised over every registered module	Every commit
3 • Golden day	A stored day (snapshots + state) produces the stored decisions exactly; changes are reviewed and re-approved deliberately	<code>brain/tests/golden/2026-09-15/</code>	Every commit
4 • Brain properties	Safety (C1, C2), monotonicity (no module off makes a decision bolder), no look-ahead, determinism, budgets	<code>brain/tests/test_properties.py</code>	Every commit

20.1 The monotonicity test

```
def test_turning_any_module_off_never_makes_decisions_bolder(golden_day):
    full = run_brain(golden_day.request, modes=ALL_ON)
    for module_id in registry.optional_modules():
        reduced = run_brain(golden_day.request, modes=ALL_ON | {module_id: Mode.OFF})
        for symbol, d in reduced.decisions.items():
            assert rank(d.word) <= rank(full.decisions[symbol].word), (module_id, symbol)
```

20.2 The safety tests

```
def test_no_trade_without_risk_gate(golden_day, fake_opinion_099):
    ctx = run_brain(golden_day.request, modes={"M07": Mode.OFF_FORCED})
    assert not [d for d in ctx.decisions.values() if d.word == "TRADE"]
    assert ctx.banner.mode == "NO_NEW_TRADES"

def test_brain_package_never_imports_brokers():
    assert_no_import("swing_trade_ml.brain", forbidden=("swing_trade_ml.brokers",
                                                         "swing_trade_ml.services.execution"))
```

20.3 Replay checks before switching a module ON

- Replay at least 60 trading days with the module ON and OFF.
- Compare TRADE count, hit rate at +8% / –4% within 15 days, and worst drawdown.
- A module is switched ON when it does not reduce hit rate and does not increase drawdown, or when it only adds information (cards, tracking) without changing decisions.

PART E • BUILD PLAN

21 • From shadow to live

The brain earns trust in four stages. Each stage has an exit test, and each can be undone with one switch.

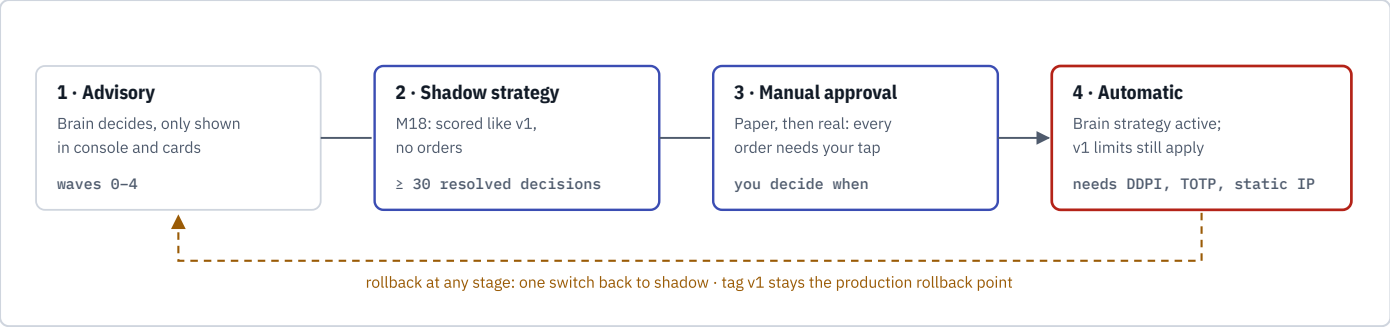


Figure 9. Stages of trust. Stage 4 also depends on the broker unlocks already known (DDPI, external TOTP, static IP).

STAGE	EXIT TEST TO MOVE ON	WHO DECIDES
1 → 2	Milestone B reached; one week of nightly runs without failed runs	Owner
2 → 3	At least 30 resolved shadow decisions; brain's hit rate and drawdown not worse than v1 over the same days (M18 comparison); no constitution test failures	Owner, with the comparison report
3 → 4	A manual-approval period the owner is comfortable with (no fixed number was set); broker unlocks in place	Owner only

What “beats v1” cannot tell us yet. The recent paper record measures a system that was changed mid-run and is not evidence either way. The comparison in stage 2 uses only the clean trial from 28 Sep 2026 onwards.

PART F · REFERENCE

22 · Risk register

Risks specific to building the brain. Business and regulatory risks of the wider platform are in the Living Architecture Review.

#	RISK	SEVERITY	MITIGATION IN THIS DESIGN
R1	No proven profit after costs	High	The brain adds structure, not edge by itself. M18 stage gates require evidence; M09 measures each module's contribution.
R2	Scope: about 100 boxes for one developer with AI help	High	Minimum brain after two waves; every later module is optional by design.
R3	A fallback path accidentally makes decisions bolder	High	Monotonic rule in the constitution and the monotonicity test on every commit.
R4	Look-ahead bias in memory or replays	High	DatedReader, stored snapshots, outcome resolved-by-as_of rule, test with future rows.
R5	Overfitting from many plug-in tilts	High	Small fixed tilts, shadow first, replay ON vs OFF before switching ON.
R6	Brain and v1 disagree and confuse the owner	Medium	Brain stays advisory until M18; the console shows both; one vocabulary.
R7	Contract changes break modules built in parallel	Medium	Contracts fixed in M00; versioned; changes need an explicit bump and owner OK.
R8	Nightly run too slow as modules grow	Medium	Per-module budgets; timings in the console; fallback on timeout.
R9	Local data is stale because Kite logins go to production	Medium	Checks use --as-of with the last stored date; backfill command.
R10	Wording reads as advice or promises (SEBI)	Medium	C12, card text lint, “Plan this trade”; RA registration handled separately.
R11	Learning proposals accepted without enough evidence	Medium	Each proposal shows its sample size and replay result; owner decides.
R12	Database growth from traces and intraday runs	Low	Retention in section 13.1; compression; housekeeping job.

PART F · REFERENCE

23 · Traceability: every box to a module

Every box of the v5 picture, what exists in code today, and where it lives in this book. Update the middle column as modules land (definition of done D9).

V5 BOX	TODAY	IN THIS BOOK
1 · Data sources		
Market data (live and historical, indices, options, futures, global)	Partly	M01, M02 · F&O and global cues later via connector
Company and fundamental data	Partly	M01 connector (NSE XBRL later) · M12
Technical indicators	Built	M02 (wraps ml/features.py)
News and events	Partly	M13 (exchange data, no scraping)
Macro and global	Partly	M10 (FII/DII, VIX) · macro calendar later
Broker and portfolio	Partly	M03, M14
User inputs and preferences	Partly	M19 (later)
Alternative data	Not built	Not planned (section 8)
Community and expert views	Not built	Not planned (section 8)
Any new source (plug and play)	Not built	M01 connector contract
2 · Input gateway		
API connectors (REST, WebSocket)	Partly	M01 (WebSocket later, same connector contract)
Streaming ingestion	Not built	Section 14 · later on measured need
Batch jobs	Built	Section 14 (existing scheduler)
File upload (CSV, Excel, JSON)	Partly	Existing imports; not a brain module
Web scraper	Not built	Not planned (section 8)
Data validation and quality checks	Partly	M01
Normalisation, enrichment, de-duplication	Partly	M01 (existing parsers and upserts)
New connector	Not built	M01 connector contract
Event bus		
Real-time event bus	Not built	Section 14 · later
Scheduled jobs	Built	Section 14 (brain jobs added)
Workflow orchestrator	Partly	M00 runner
Priority handling	Not built	Section 14.1
Error handling and retry	Partly	M00 (fallbacks), section 14.1
Scalable, add new flows	Partly	M00 registry
3 · Brain		
3.1 Perception	Partly	M01, M02
3.2 State engine	Partly	M03 (+M10, M11, M14)
3.3 Situation recognition	Partly	M04 (+M12, M13)
3.4 Memory and experience	Partly	M05
3.5 Reasoning and models	Partly	M06
3.6 Risk and safety	Partly	M07
3.7 Decision engine	Partly	M08 (two-set vocabulary)
3.8 Learning	Partly	M09 (proposals only)
Plug-ins		
Market brain	Partly	M10
Sector brain	Partly	M11
Stock brain	Not built	M12
News brain	Not built	M13
Portfolio (optimisation) + Portfolio brain	Not built	M14 (merged into one)

Strategy brain	Partly	M06 (source reliability by regime)
User brain	Not built	M19 (later)
Risk brain	Built	M07
New module	Not built	Section 10 (module contract)
Knowledge base		
Patterns library, strategy rules, playbooks	Partly	M12 setups, M04 labels, M10 mode rules
Trained models, model registry	Built	Existing; used by M06
Feature store	Not built	M02 feature_snapshots
Experience DB	Partly	M05 brain_experience, brain_episodes
Version control	Partly	Manifest versions, contract versions, model registry
Constitution		
Risk limits	Built	M07 · C3
Compliance rules (SEBI)	Not built	C12 wording; RA registration outside this book
Data integrity	Partly	M01 · C5
Bias checks	Partly	NFR-07, DatedReader, M09 calibration
Uncertainty handling	Not built	M06 uncertain flag → WAIT
Human override	Built	C8, M17 (halt, switch, overrule)
Audit logs	Partly	brain_runs, brain_decisions, trace
No uncontrolled changes	Partly	C9, P12, module modes
5 • User experience		
5.1 Signal and suggestion	Partly	M08, M16 (“Plan this trade”)
5.2 After investing	Partly	M15, M16
5.3 Expected returns and tracking	Not built	M05 evidence, M15 band
5.4 Unexpected situation	Not built	M15 (stop hit reported as hit, C7)
5.5 Exit intelligence	Partly	M15 + existing exit policy
6 • Feedback loop		
Track outcomes	Built	M09 (+ v1 evaluation jobs)
Analyse why	Not built	M09 failure investigation
Update memory	Not built	M05 nightly update
Retrain models	Partly	M09 proposal → challenger
Benchmark comparison	Partly	M09, M18 comparison
Report insights	Partly	M09 weekly report
Continuous (new rules, data)	Not built	M09 proposals with approval
7 • Infrastructure		
Databases	Built	PostgreSQL + TimescaleDB only (NFR-12)
Cache	Built	Redis (run lock)
Storage and streaming	Not built	Later on need
Cloud	Built	DigitalOcean, portable
Vector DB	Not built	pgvector if M05 needs it
Message queue	Not built	Redis first, later on need
Compute (Kubernetes, Docker)	Built	Docker Compose; no Kubernetes
Broker integrations	Built	v1 only; brain never imports brokers (C10)
External APIs	Partly	M01 connectors
8 • Observability		
System health	Partly	M17
Data freshness	Partly	M01, M17

Incident detection	Partly	M17 (failed runs alert)
Auto recovery	Partly	M00 fallbacks, section 14.1 retry
Capacity and scaling	Not built	NFR-04 timings; later
Audit, security, access	Partly	NFR-13, M17 owner-only
9 • Extensibility		
Data source, indicator, rule plug-ins	Not built	M01 connectors, section 10 modules
Strategy plug-ins	Built	Existing registry; BrainStrategy (M18)
User type, UI component plug-ins	Partly	Plans branch; M16 components
Channel plug-ins	Not built	M16 alert senders
Feature flags, A/B (shadow)	Partly	Module modes ON/SHADOW/OFF, M18
New business modules	Not built	Not planned in year one
10 • Flexibility		
Add sources, models, strategies anytime	Partly	Sections 10, M01
Replace components without breaking	Partly	Contracts (section 12)
Feature flags, shadow, gradual rollout	Partly	Module modes, section 21
Clear contracts	Partly	Section 12, contract versions
Future modules	Not built	Not planned (section 8)
Versioning and safe upgrades	Partly	NFR-10, tag v1 rollback
4 • Outputs		
Stock signals	Partly	M08, M16
Personalised recommendations	Not built	M19 sizing only (later)
Real-time alerts	Partly	M16
Portfolio actions	Not built	M08 holdings words, M14 notes (never executed for users)
Day-by-day tracking	Partly	M15
AI copilot and insights	Not built	Later: reads brain traces; separate design
Reports and scorecards	Partly	M09 report, M18 comparison
User interfaces	Partly	M16, M17
Broker execution (optional)	Not built	Owner only via M18; for users not planned
Institutional use	Not built	Not planned (section 8)

103 boxes traced. “Today” is from the Living Architecture Review, checked against the code on 29 Sep 2026.

PART F • REFERENCE

24 • Glossary

TERM	MEANING
ATR	Average true range: the typical daily price movement of a stock, used to size entry zones and bands.
Band	The range (25th to 75th percentile) of where similar past trades were on each day.
Barrier outcome	Which came first within 15 trading days: +8% (target) or −4% (stop). If neither, the time limit.
Breadth	The share of stocks that are rising or above their averages. Weak breadth means few stocks are carrying the index.
Calibration	Whether a model's 60% really wins about 60% of the time.
Constitution pass	The final safety check over a whole run inside M00; it cannot be switched off.
Contract	The fixed shape of data passed between modules, with a version like Opinion@1.
DatedReader	The only way modules read the database; it hides anything after the run's time.
DEFENSIVE	Market mode with fewer, smaller new trades.

Drift	A trade below the expected band but above its stop; or model inputs moving away from what it was trained on.
Fallback	The simple, safe answer a step gives when its module is off or failing.
FII / DII	Foreign and domestic institutional investors; their daily net buying or selling.
Golden day	A stored day of inputs whose decisions are kept as a reference for tests.
Hysteresis	Requiring a condition to hold twice before switching back, so a mode does not flip-flop.
Manifest	A module's label: id, step, version, what it reads and writes, time budget.
Monotonic rule	Missing modules or data can only make a decision more cautious.
Opinion	One source's view on a stock: a stance from -1 to +1, a confidence and reasons.
Plug-in	A module that adds knowledge to a step without owning it.
Shadow	Runs and records, but nothing acts on it.
Situation	A named market or stock condition, like “pullback in an up-trend”.
Trace	The record of which modules ran, fell back or were skipped in a run, and why.
v1	Today's running bot; tag v1 = 64fcb05 is the production rollback point.

Document history

VERSION	DATE	CHANGE
1.0	29 Sep 2026	First version, from the v5 picture, the Living Architecture Review and the settled decisions.